TECHNICAL HANDBOOK

FLOD_MAS Handbook

Architecture and pipelines of the glue-line inspection system

Built	2026-10-12
From	branch `tooling/pipeline-lab`, commit `0e98260`
Covers	the inspection application and the glue-line research on that branch, and the application on `main` as read at the commit named in Part D
Sources	`docs/handbook/` (chapters, diagrams, build script)

Contents

PART A - OVERVIEW

1 Purpose and scope
- 1.1 What the system does
- 1.2 What is known about the glue line
- 1.3 What this handbook covers
- 1.4 How far each statement can be trusted

2 System at a glance
- 2.1 Who and what the software talks to
- 2.2 The idea in five steps
- 2.3 Colour key of the diagrams
- 2.4 Numbers worth remembering

3 Reading guide and glossary
- 3.1 Where to start
- 3.2 Glossary
- 3.3 Conventions

PART B - INSPECTION APPLICATION (THIS BRANCH)

4 Module architecture
- 4.1 Layers
- 4.2 Modules and what each is for
- 4.3 What is not part of the application

5 Runtime flow of one inspection
- 5.1 From a frame to the result tabs
- 5.2 The hand-cycle checkpoint
- 5.3 Threads and what passes between them
- 5.4 Statuses a fabric can end with

6 Segmentation providers
- 6.1 One contract
- 6.2 The three providers on this branch
- 6.3 How the local detector works, in short
- 6.4 Switches that keep old behaviour

7 Measurement and grading
- 7.1 From a polygon to a verdict
- 7.2 Optional edge refinement
- 7.3 Millimetres
- 7.4 Grading
- 7.5 What is saved for each fabric

8 Calibration and millimetres
- 8.1 From a board to a millimetre
- 8.2 Limits that matter for the glue line
- 8.3 State on the development PC

9 Configuration and saved data
- 9.1 Which module reads which settings
- 9.2 The settings that shape an inspection
- 9.3 A capture set on disk

PART C - GLUE-LINE DETECTION RESEARCH (THIS BRANCH)

10 The object and why it is hard
- 10.1 What the glue line looks like
- 10.2 The signal, in numbers
- 10.3 What makes it hard
- 10.4 Images used

11 How the flow evolved
- 11.1 Versions and the verdict on each
- 11.2 The same line through versions 2 to 5
- 11.3 What each step taught
- 11.4 What was tried and does not work

12 The detection flow in use, stage by stage
- 12.1 The whole flow
- 12.2 One frame through the main stages
- 12.3 Stage 1: panel region
- 12.4 Stage 2: rough line
- 12.5 Stage 3: SAM with point prompts
- 12.6 Stage 4: leak repair
- 12.7 Stages 5 to 8: states, band edges, ribbon
- 12.8 What the flow gives on the sample set

13 Speed: the local GPU flow
- 13.1 Where the time goes
- 13.2 What made the difference
- 13.3 Proof that it is offline
- 13.4 Not done

14 Accuracy: the truth bench
- 14.1 How the bench works
- 14.2 Edge methods compared
- 14.3 Precision on real repeat shots
- 14.4 Width defects and false alarms
- 14.5 In millimetres

15 The trained segmenter (SegFormer and CLAHE)
- 15.1 Training and use
- 15.2 The real test: frames of other pieces, never trained on
- 15.3 CLAHE
- 15.4 Against SAM
- 15.5 State

16 Width in millimetres and width changes
- 16.1 The method
- 16.2 Why the mask width, and why not three sigma
- 16.3 What it reaches
- 16.4 Segmentation glitches
- 16.5 What it does not cover

17 Research tools
- 17.1 The map

17.2 The Pipeline Lab
17.3 Commands
17.4 What lives outside the repository
17.5 Tests

PART D - THE APPLICATION ON MAIN (READ-ONLY)

18 Architecture of the application on `main`
18.1 What changed against the common base
18.2 Settings added or changed
18.3 The updater

19 The inspection pipeline on `main`
19.1 From a photo to graded segments
19.2 How long it takes
19.3 Review findings
19.4 Trigger, queue and reset
19.5 What is saved
19.6 Compared with the application on this branch

20 The two detection pipelines compared
20.1 Side by side
20.2 Time on the same frames
20.3 Measured on the same 74 frames
20.4 The one combination that was tested
20.5 What merging would involve

PART E - STATE OF THE WORK

21 Decision guide
21.1 Which method for which need
21.2 The options in one table
21.3 Rules that held throughout

22 What is verified, what is not, and next steps
22.1 Verified by measurement on the sample frames
22.2 Judged by the user
22.3 Not verified
22.4 Next steps, in order of what they unlock
22.5 Rules for changing this system

23 Appendices
23.1 File index (branch `tooling/pipeline-lab`)
23.2 Documents
23.3 Result pages (outside the repository)
23.4 Environments
23.5 Rebuilding this book

PART A

Overview

1 Purpose and scope

1.1 What the system does

FLOD_MAS inspects the glue line laid on fabric panels of a garment. An operator places panels on a table under a camera and takes their hands away; the software photographs the table, finds the glue line on every panel, measures how wide it is along its length and shows a verdict for each panel. The one defect it looks for is **width out of range**.

The application is a Tkinter desktop program for a Windows inspection PC (Python 3.12). A Linux PC with a CUDA graphics card is used for development and for the experiments this handbook describes.

1.2 What is known about the glue line

These facts were stated by the user or measured on the sample frames; the rest of the book leans on them.

Fact	Source
The glue is a thin, glossy bead: a darker band with a bright core that shows in places	user; measured
The normal strip is **4 mm** wide most of the time	user, 2026-10-08
The camera is **not calibrated**, so millimetre values may be off	user
The supplied sample strips have **no defects**	user
The only defect is width out of range	user
The glue always lies inside the panel region that the region model marks; cardboard is not glue	user
The same line is shiny in one place and matte in the next	user; measured
A 4 mm strip reads about 30 px in a 4608 x 3456 frame: about 7.6 px per mm	measured, 257 lines
On pale fabric the band is only about 7 grey levels darker than the fabric; on dark fabric 43-50	measured
The target is to resolve width changes of 0.1-0.2 mm	user, 2026-10-09

1.3 What this handbook covers

There are two lines of code that grew from one base and have **not** been merged. The book describes both and keeps them apart.

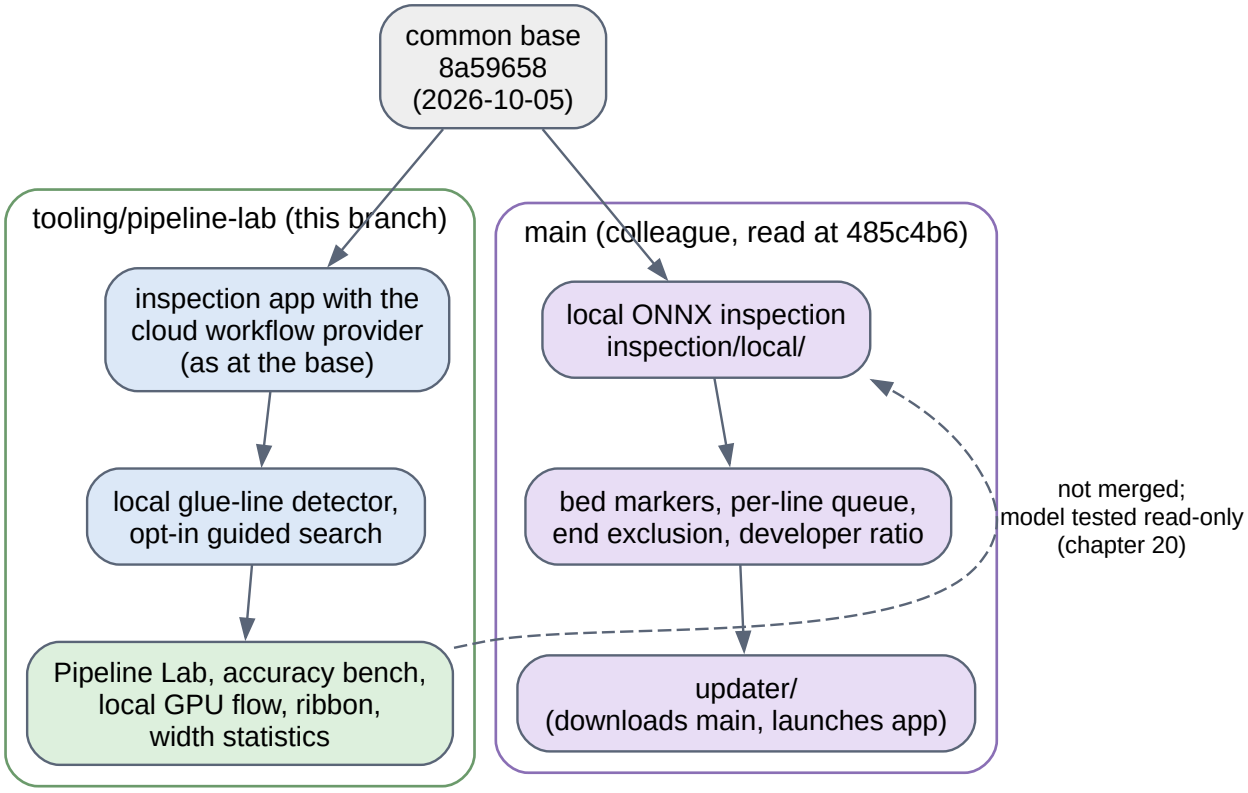


Figure 1.1 The two lines of code. Parts B and C describe the left one; Part D the right one.

Part	Subject	Branch
A	Overview, reading guide	both
B	The inspection application: modules, runtime flow, providers, measurement, calibration, settings	`tooling/pipeline-lab`
C	Glue-line detection research: the detection flow in use, speed, accuracy, trained segmenter, width statistics, tools	`tooling/pipeline-lab`
D	The colleague's application: its architecture and pipeline, and how the two compare	`main`, read at commit `485c4b6`
E	Decision guide, what is verified and what is not, appendices	both

1.4 How far each statement can be trusted

The book uses three words on purpose.

- **Measured**: a number produced by running code on the sample frames; the table it comes from is in `docs/glue-line/GLUE_LINE_FINDINGS.md`.
- **Read**: taken from the source code without running it. All of Part D's description of the application on `main` is read; only its glue model was run (chapter 20).
- **Not verified**: stated as such, and collected in chapter 22.

> **No ground truth exists.** No line has been measured with a caliper and no defective strip has been supplied. Accuracy figures are against painted glue lines (a model of the real thing) and against agreement between methods; whether any method catches a real width defect is not known.

2 System at a glance

2.1 Who and what the software talks to

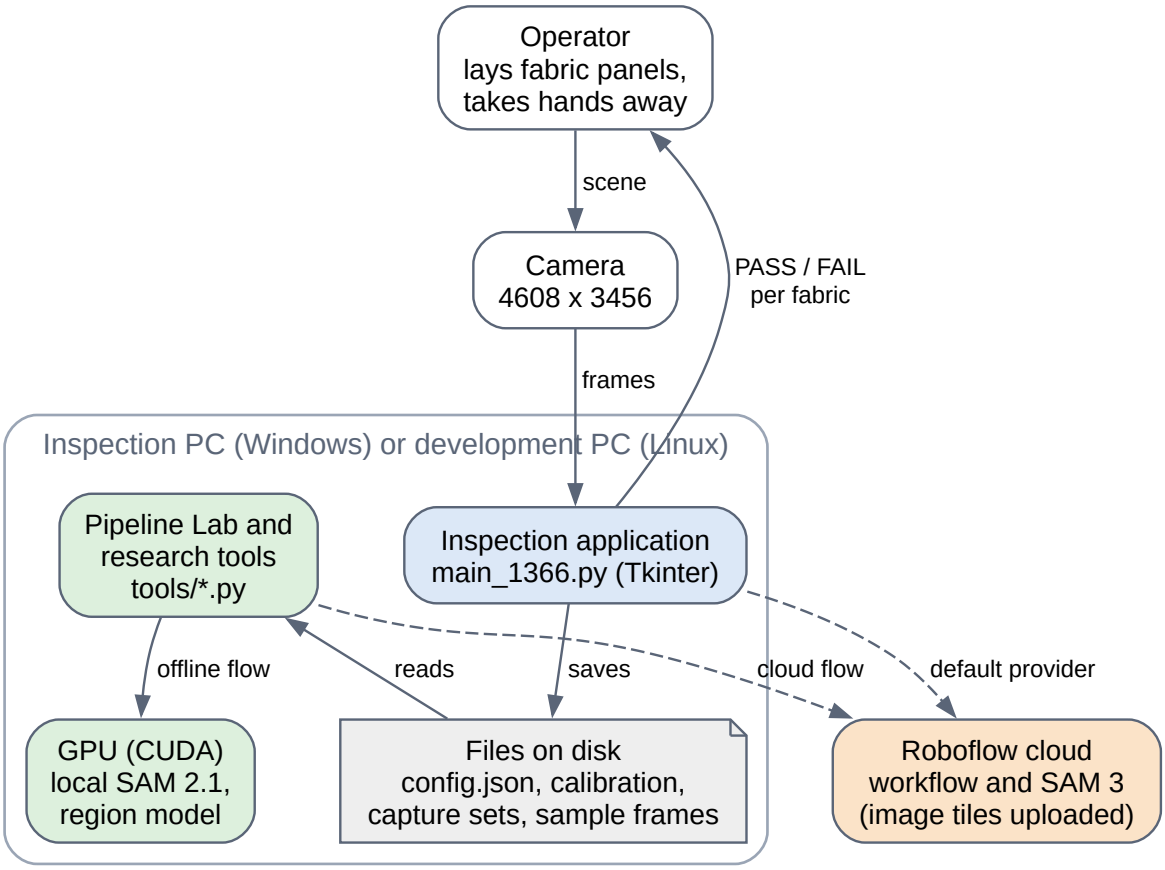


Figure 2.1 System context. Solid arrows are always present; dashed arrows exist only when a cloud provider is chosen.

- **The inspection application** owns the camera, the trigger, the result screen and the saved capture sets. With the default settings on this branch it sends a reduced copy of each capture to a Roboflow workflow, which returns the strip outlines.
- **The research tools** never touch the camera. They read saved frames, run the detection flow on them and write overlays, numbers and reports. They can run entirely offline on the GPU.
- **Files on disk** join the two: `config.json`, the saved calibration, and the capture sets.

2.2 The idea in five steps

Every pipeline in this book, on either branch, is a variation of the same five steps.

Step	Question it answers	This branch, app	This branch, research (version 5)	`main`
1	Where can glue be?	whole frame, or `segmentation.roi`	panel-region model (U-Net)	the same U-Net
2	Where does each line run?	the cloud workflow	rough line by the local detector	curved side of each region + 100 px band
3	Which pixels are glue?	SAM 3 in the workflow	SAM 2.1 with point prompts, repaired	SegFormer-B0 on the unfolded band
4	Where exactly are the two edges?	the polygon as returned (optional edge refinement)	band edges by trained trees, fitted as a smooth ribbon	the cleaned mask's boundary
5	How wide, and is that acceptable?	ten segment averages in mm against target and tolerance	width along the line; deviation rule; width statistics	ten segment averages in mm against target and tolerance

2.3 Colour key of the diagrams

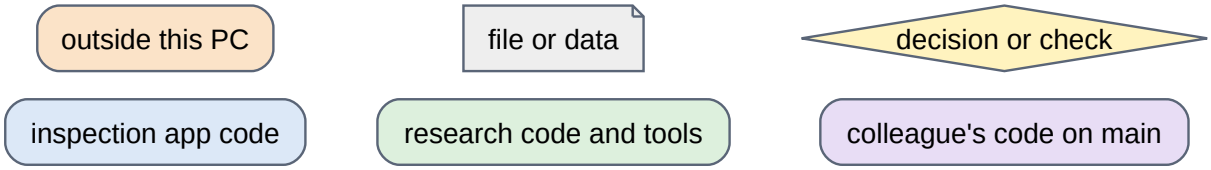


Figure 2.2 Colours and shapes used in every diagram.

2.4 Numbers worth remembering

Quantity	Value	Chapter
Frame size of the sample set (M1 captures)	4608 x 3456, 74 frames, 257 glue lines	10
Width of the strip in the picture	about 30 px = 4 mm (7.6 px per mm, uncalibrated)	16
Time per frame, cloud flow (version 4)	22 s dark, 45 s pink, 23 s white	13
Time per frame, local GPU flow (version 5)	about 5 s, nothing uploaded	13
Time per frame, glue model on `main`	3 s on the CPU	20
Edge error against painted truth, version 5	0.5-0.7 px RMS (about 0.07-0.09 mm)	14
Smallest width change found, dark fabric	0.14 mm if it lasts 42 mm; 0.36 mm if it lasts 5 mm	16
Smallest width change found, pink and white	0.35-0.76 mm	16

3 Reading guide and glossary

3.1 Where to start

If you want to...	Read
understand how the application is built	chapters 4-6
follow one inspection from frame to verdict	chapter 5, then 7
change a setting	chapter 9, then `docs/inspection/CONFIGURATION.md`
know how the glue line is found in the research flow	chapters 10-12
know how fast and how accurate it is	chapters 13 and 14
know how width changes are judged statistically	chapter 16
run a research tool	chapter 17 and appendix 23
understand the application on `main`	chapters 18-19
decide which method to use	chapter 21
know what has not been proven	chapter 22

3.2 Glossary

Term	Meaning
Panel, fabric	One piece of fabric on the table. A frame holds two to four.
Strip, glue line, bead	The line of glue on a panel. "Bead" is used for the glue seen in cross-section.
Band	The darker stripe the bead makes on the fabric, about 30 px (4 mm) wide. Its two edges are what is measured.
Core	The bright, shiny middle of the bead, visible only in places.
ROI, panel region	The part of a panel where the glue can be, marked by the region model (a U-Net).
Rough line	A first estimate of where a line runs, good to about 20 px, from the local detector.
SAM	Segment Anything Model. SAM 3 runs in the Roboflow cloud; SAM 2.1 runs on the PC's GPU. It is given points on the line and returns a mask.
Leak, spill	A place where SAM's mask runs out of the bead into the fabric beside it.
Straightened strip	The picture resampled along a line so that the line becomes straight; edges are then read across it.
State	What the picture is like at one place along a line: shiny, matte, one-sided, weak, none.
Trees	Gradient-boosted decision trees (LightGBM) trained to correct an edge position from the picture around it.
Ribbon	The final outline: a smooth centre line with a slowly varying width, fitted to all the evidence.
Mask	A picture-sized map saying, per pixel, glue or not (or a probability of glue).
SegFormer	A transformer segmentation network. Used as a trained glue segmenter in the research (B2) and on `main` (B0).
CLAHE	Contrast-limited adaptive histogram equalisation: a local contrast boost.
Glitch	A stretch where the mask itself is in doubt, so a width change read there may not be real.
Provider	A class that turns a frame into strip polygons for the application; chosen in `config.json`.

Term	Meaning
Capture set	The folder of files saved for one inspection.
Opt-in	A setting whose shipped value keeps the old behaviour. All new behaviour on this branch is opt-in.

3.3 Conventions

- File paths are relative to the repository root unless they start with `~`.
- `~/flod_experiments/` holds experiment scripts, trained models and result pages. It is outside the repository on purpose.
- Pixels always mean pixels of the full camera frame unless said otherwise.
- "Dark", "pink" and "white" are the three fabric colours in the sample set.

PART B

Inspection application (this branch)

4 Module architecture

This part describes the application as it is on `tooling/pipeline-lab`. It is the application of the common base plus an opt-in local detector; the colleague's later rewrite is in Part D.

4.1 Layers

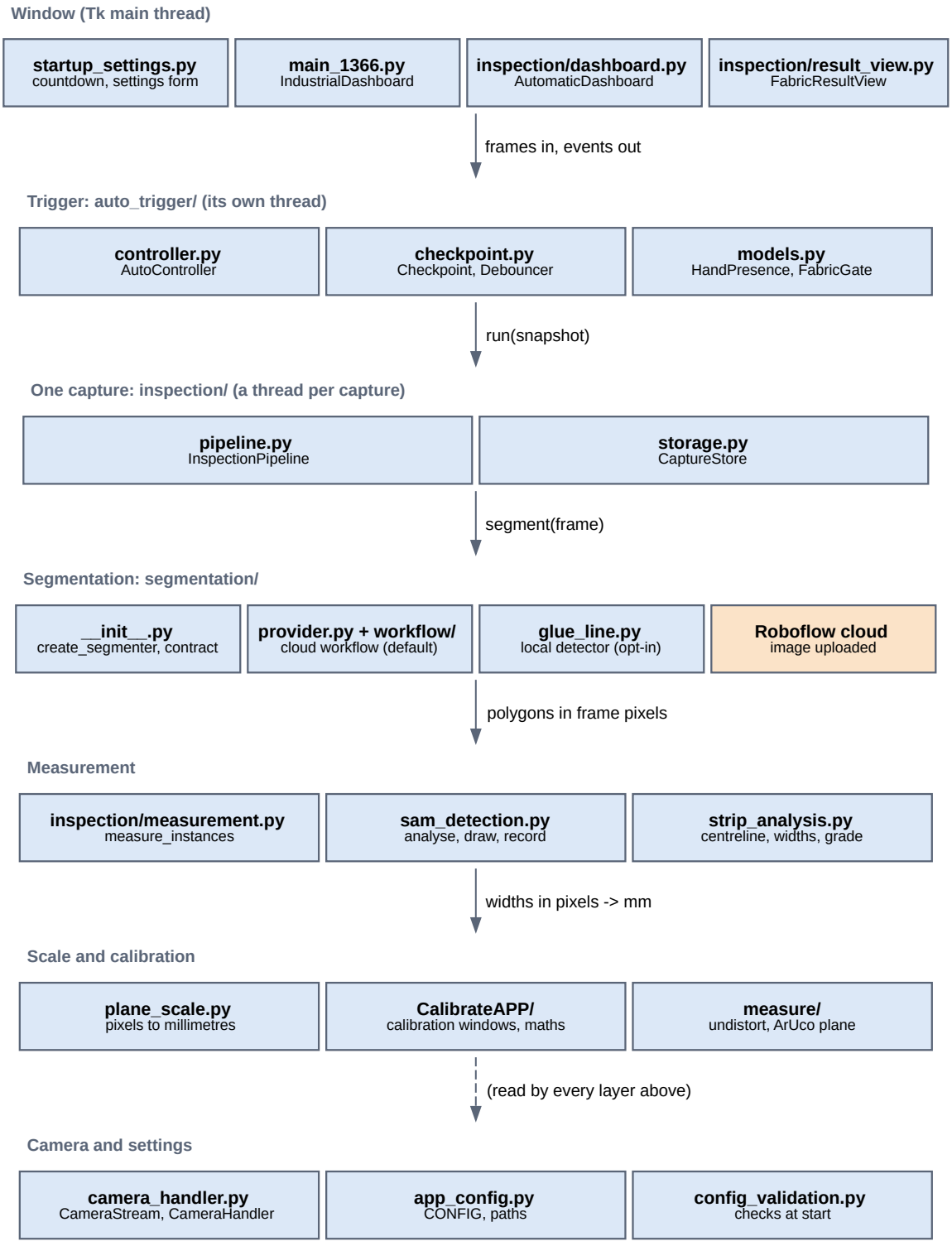


Figure 4.1 The application by layer. An arrow from one layer to the next means "calls" or "passes to".

The design keeps three things apart:

- **Tk widgets are touched by one thread only.** Everything slow runs elsewhere and reports back through a queue of events.
- **Segmentation is behind one contract** (chapter 6). The pipeline does not know whether the outline came from the cloud or from a local detector.
- **Measurement is the same code for every provider.** It needs only a polygon in frame pixels.

4.2 Modules and what each is for

File	Main names	Job
`main_1366.py`	`IndustrialDashboard`	The window: live view, bottom panel, settings, size and tolerance profile. Inherits the automatic behaviour from `AutomaticDashboard`.
`startup_settings.py`	`StartupWindow`	Five-second countdown at start; **S** opens the settings form. Runs before any module that caches settings.
`app_config.py`, `config_validation.py`	`CONFIG`, `validate_config`	Load `config.json`; reject invalid settings at start.
`camera_handler.py`	`CameraStream`, `CameraHandler`	Background capture at full resolution; keeps the newest frame; applies rotation and optional fixed exposure.
`measure/makeUndistored.py`	`ImageUndistorter`	Removes lens distortion with the saved calibration.
`inspection/dashboard.py`	`AutomaticDashboard`	Starts the controller, polls its events every 100 ms, shows results.
`auto_trigger/controller.py`	`AutoController`	A thread: hand detection, debounce, checkpoint, fabric gate, starting a capture, protection against stale results.
`auto_trigger/checkpoint.py`	`Checkpoint`, `Debouncer`	The hand-cycle state machine.
`auto_trigger/models.py`, `fabric_check.py`	`HandPresence`, `FabricGate`	MediaPipe hand landmarker on a 320 px wide copy; Keras fabric classifier.
`inspection/pipeline.py`	`InspectionPipeline`	One capture: segment, validate, measure, save.
`segmentation/__init__.py`	`create_segmenter`, `Instance`, `SegmentationResult`, `validate_result`	The provider contract.
`segmentation/provider.py`, `segmentation/workflow/`	`WorkflowSegmenter`	The cloud workflow provider (default).
`segmentation/glue_line.py`, `glue_shape.py`	`GlueLineSegmenter`	The local detector (opt-in), with the optional shape condition and region-guided search.
`inspection/measurement.py`	`measure_instances`, `FabricResult`	Per fabric: crop, measure, grade, annotate.
`sam_detection.py`	`analyze_detection`, `draw_analysis`, `strip_record`	Glue between a polygon and the strip analysis; drawing; the saved record.

File	Main names	Job
`strip_analysis.py`	`analyze_ring`, `refine_widths`, `to_metric`, `grade_widths`	Centreline, widths, segments, millimetres, grading.
`plane_scale.py`	`load_frame_scale`, `PlaneScale`	Pixels to millimetres on the calibrated plane.
`CalibrateAPP/`	`CalibrationApp`, `calibration_math`	Calibration windows and their mathematics.
`inspection/storage.py`	`CaptureStore`	One folder per capture; retention limit.
`inspection/result_view.py`, `crop_result_view.py`	`FabricResultView`	Result tabs with zoom and pan.
`app_storage.py`	`data_path`	Where user data lives (Documents / `data files`), apart from code.
`tools/replay_capture.py`		Runs saved images through the same pipeline, offline.

4.3 What is not part of the application

Everything in chapter 17 (the Pipeline Lab and the research modules `sam_refine`, `sam_local`, `bead_edges`, `ribbon`, `bead_ml`, `edge_methods`, `synthetic_bead`, `width_stats`, `segformer_local`) is used only by the tools. The application does not import them, with one exception: `glue_line.py` imports `sam_refine` when the opt-in key `sam_refine` is turned on.

5 Runtime flow of one inspection

5.1 From a frame to the result tabs

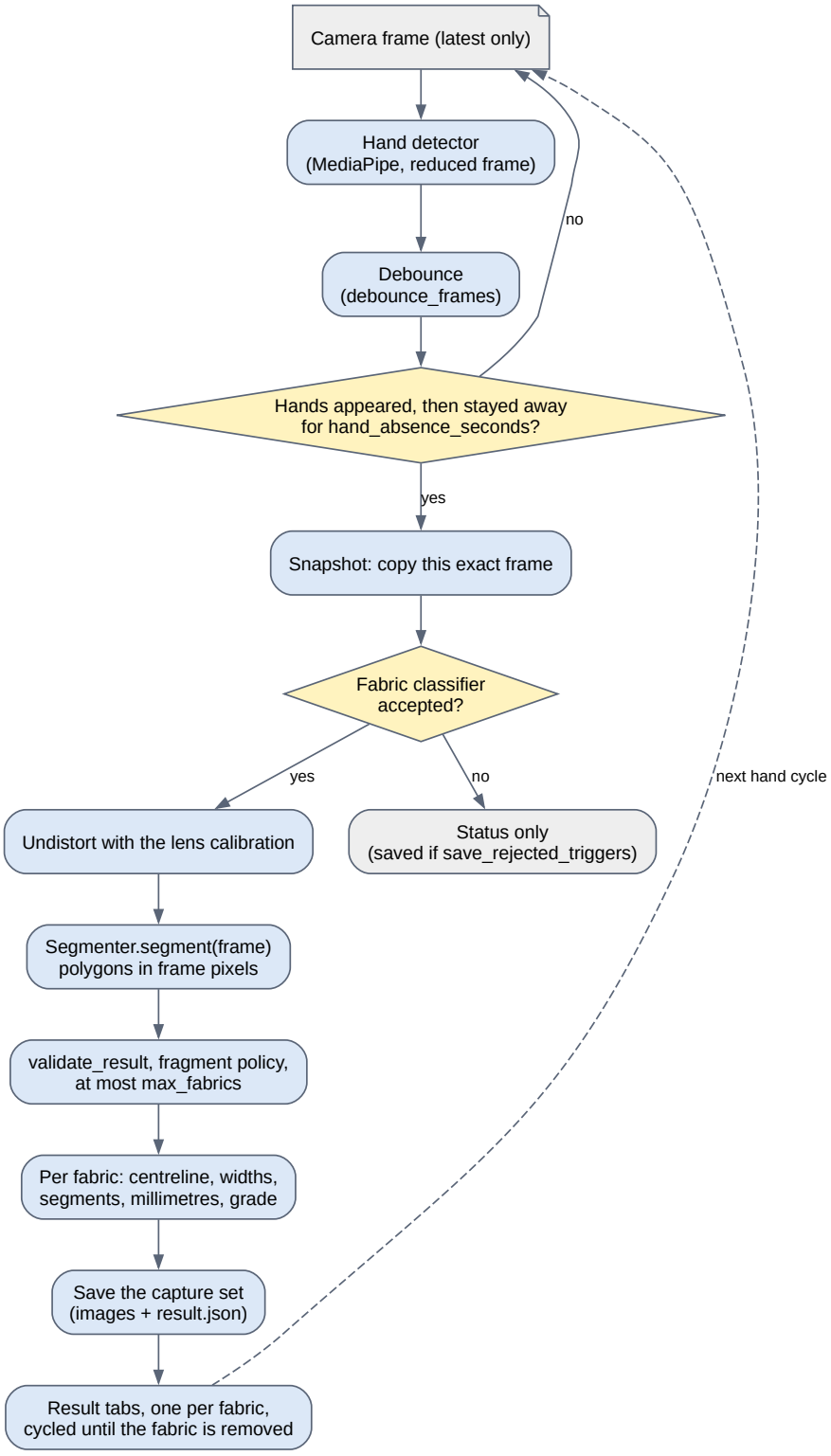


Figure 5.1 One inspection cycle. The loop on the left runs on every new frame; the column on the right runs once per capture.

1. **Hands.** Each new frame is reduced to 320 px wide and given to the hand detector. The raw yes/no is debounced over `debounce_frames` frames.

2. **Checkpoint.** A capture is triggered when hands were seen and then stayed away for `hand_absence_seconds`.
3. **Snapshot.** That exact frame is copied. The hand result, the fabric verdict, the saved image and the measurement all refer to this one copy.
4. **Fabric gate.** A classifier labels the snapshot `full_fabric`, `half_fabric` or `no_fabric`. Only labels in `accepted_labels` above `fabric_min_confidence` go on; `no_fabric` clears the screen.
5. **Undistort**, then **segment** through the provider, then **validate**: polygons must be inside the frame; with `fragment_policy = "error"` several pieces for one fabric stop the capture; more than `max_fabrics` strips stop it.
6. **Measure** every fabric (chapter 7), **save** the capture set (chapter 9), **show** one tab per fabric, cycling every `result_display_seconds` until the fabric is removed.

5.2 The hand-cycle checkpoint

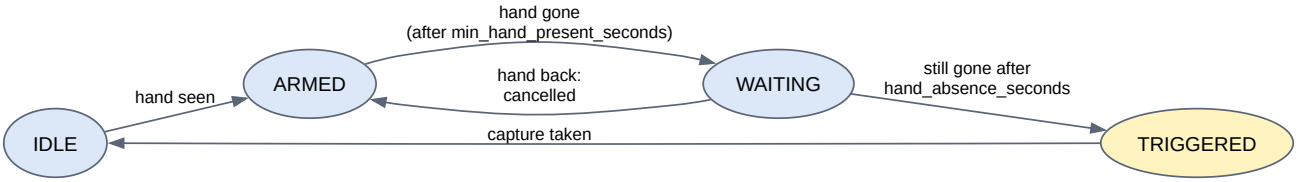


Figure 5.2 States of the checkpoint. A capture is taken on entering TRIGGERED.

A new hand seen while work is in progress raises the controller's *generation* number at once, before debouncing. Results carry the generation they were started under; a result from an older generation is saved but not shown. This is how a result for an arrangement the operator has already changed is kept off the screen.

5.3 Threads and what passes between them

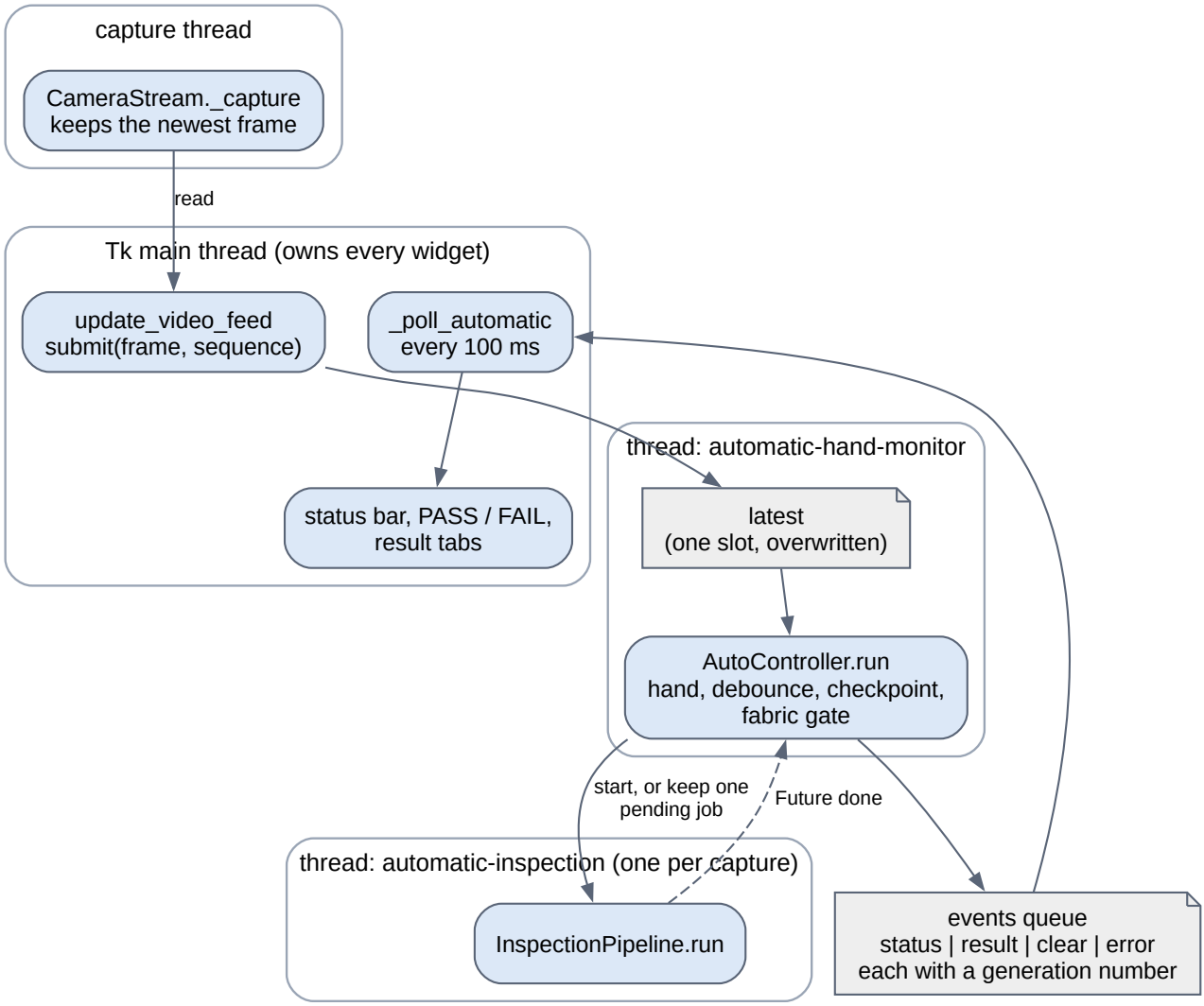


Figure 5.3 Threads. Only the Tk thread touches widgets; the others talk to it through the events queue.

Thread	Does	Hands over
Capture	Reads the camera, keeps the newest frame	frame to the Tk thread
Tk main	Draws; calls `submit(frame, sequence)`; polls events every 100 ms	newest frame into a one-slot box
`automatic-hand-monitor`	Hand, checkpoint, fabric gate	a job to the inspection thread; events to Tk
`automatic-inspection`	`InspectionPipeline.run` for one capture	a finished `Future`

- The one-slot box means the monitor always works on the newest frame; older ones are dropped, never queued.
- At most one capture is being processed and one is waiting. A newer capture replaces the waiting one, which is recorded as `superseded`.
- Events are `status`, `result`, `clear` and `error`. Errors are always shown, even from an outdated capture.

5.4 Statuses a fabric can end with

Status	Meaning	Counts as
`PASS`	every segment's average width is inside target +- tolerance	pass
`FAIL`	at least one segment is outside	fail
`UNMEASURED`	no millimetre calibration: pixels only	warning
`NO STRIP`	a fabric was matched but no strip came back (`report_missing_strips`)	warning
`FRAGMENTED`	the strip came in pieces; the largest is shown (`fragment_policy = "largest"`)	warning
`LOW CONFIDENCE`	below `segmentation.min_confidence`	warning

The screen shows FAIL if any fabric fails, PASS only if all pass and nothing warned, and WARNING otherwise. A fabric that could not be graded never reads as a pass.

6 Segmentation providers

6.1 One contract

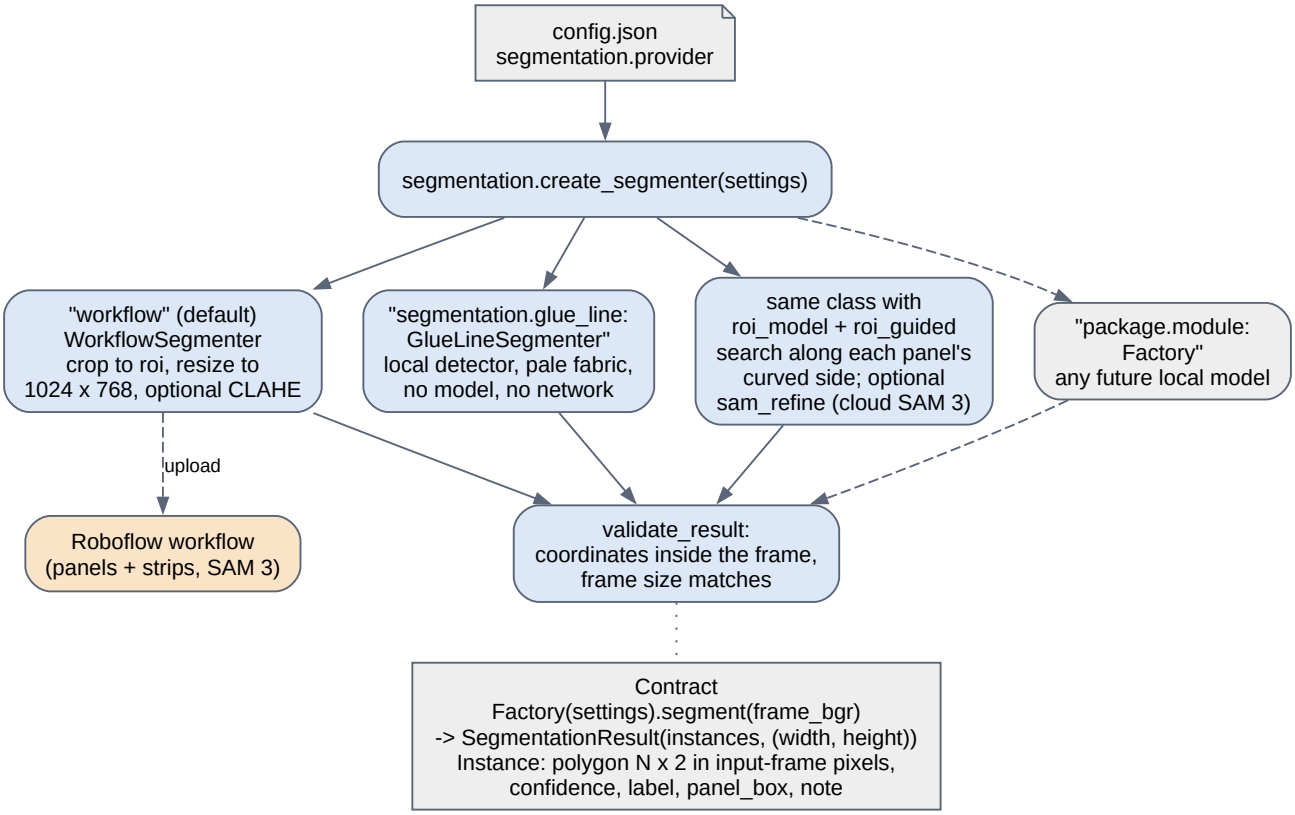


Figure 6.1 The provider contract. Everything a provider does to the image stays inside it; only frame-pixel polygons come out.

A provider is any class with `__init__(settings)` and `segment(frame_bgr)` that returns a `SegmentationResult`. Its `Instance` objects carry a polygon in the pixels of the frame it was given, a confidence, a label, an optional panel box and an optional note. Resizing, colour conversion, model loading and post-processing are the provider's own business; `validate_result` refuses anything that is not in frame coordinates.

`segmentation.provider` in `config.json` chooses it: the word `workflow`, or `package.module:Factory` for any other class.

6.2 The three providers on this branch

	Cloud workflow (default)	Local detector	Region-guided local detector
Setting	`"workflow"`	`"segmentation.glue_line:GlueLineSegmenter"`	the same, with `roi_model` and `roi_guided`
Where it runs	Roboflow (image uploaded)	this PC, CPU	this PC; SAM 3 in the cloud only if `sam_refine`
Input	`roi` crop resized to 1024 x 768, optional CLAHE	full-resolution frame	full-resolution frame + panel regions
Finds	panels and strips, any fabric	the bead on pale fabric: a bright line between two dark lines	one line along the curved side of each panel
Dark fabric	yes	reports nothing	yes (`bead_model = "line"` or `"auto"`)
Needs	`ROBOFLOW_API_KEY`, a connection	nothing	the region model file (not in the repository)
Measured	by the reference project	56 pale sample frames: 52 beads, 7-30 s per frame	through the Pipeline Lab: chapters 12-14

6.3 How the local detector works, in short

1. Remove the lighting gradient; run a long directional line filter, kept separately for dark lines and bright lines.
2. Look only inside pale fabric, away from its outline.
3. At each point of a dark line, look straight across for a second dark line running the same way within the width range, with a bright line between them.
4. Join such points into centrelines; bridge short faded stretches (`join_gap_px`, `track_gaps`).
5. Keep a strip only where the centre is brighter than just outside its edges.
6. Optionally (`shape_prior`) require the line to have the product's known wave, which reports whole lines and rejects seams and folds.

The region-guided mode replaces steps 2-5 by a search inside a corridor along the curved side of each panel region (`corridor_near_px`, `corridor_far_px`). It is what the research flow uses for its *rough line*.

6.4 Switches that keep old behaviour

Every key added on this branch is off or neutral as shipped; with the shipped values the detector's output was checked byte for byte against the version before them.

Key in `segmentation.glue_line`	Shipped	Turns on
`shape_prior.enabled`	false	the wave-shape condition
`background`	`white`	fabric on any table (`any`)
`bead_model`	`edges`	one-band model for coloured fabric (`line`, `auto`)
`roi_model`	empty	the panel-region model
`roi_guided`	false	one line per panel side
`roi_strict`	false	never search outside the region
`enhance_bead`	false	search on the bead-enhanced image
`sam_refine`	false	outline with SAM 3 (**uploads image tiles**)

The full tables are in `docs/inspection/AUTOMATIC_INSPECTION.md`.

7 Measurement and grading

7.1 From a polygon to a verdict

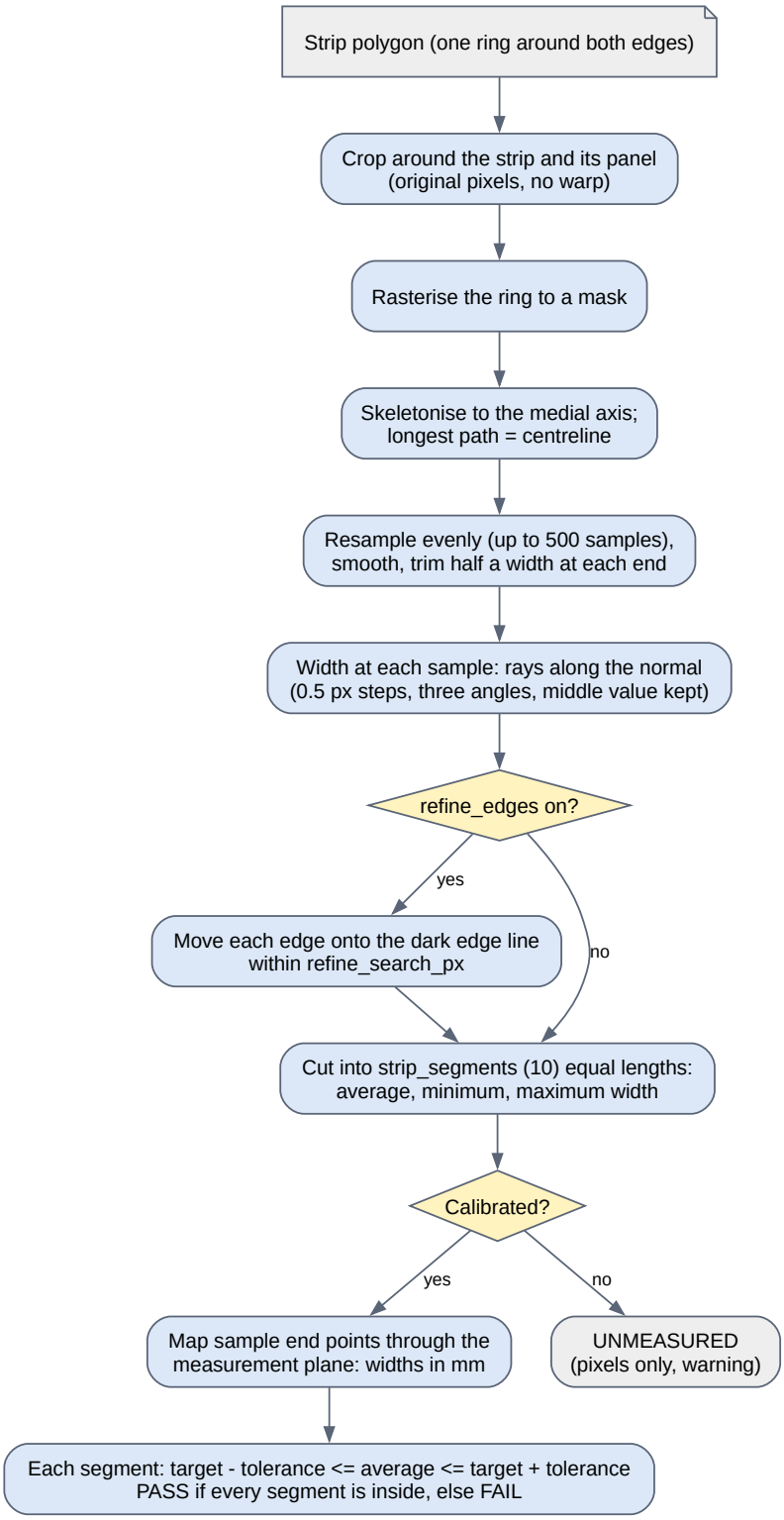


Figure 7.1 The measurement of one strip. The same code runs for every provider.

A provider returns one ring that traces both edges of the strip. A ring alone says nothing about where the strip runs or how wide it is, so the analysis rebuilds both.

Step	What is done	Value
Mask	The ring is filled	a strip under 40 px in area is a speck and is dropped
Centreline	Skeleton of the mask; its longest path	under 30 px is too short
Resample	Evenly along the path, then smoothed	up to 500 samples; smoothing 2 % of the length
Ends	Half a strip width is trimmed from each end	so the rounded tip is not measured
Width	A ray marches along the normal both ways until it leaves the mask	0.5 px steps
Skew guard	Measured at three small angles; the middle value is kept	+-0.10 rad
Segments	The line is cut into equal lengths	`sam_detection.strip_segments` = 10
Per segment	Average, minimum and maximum width	

7.2 Optional edge refinement

With `sam_detection.refine_edges` on, each edge is moved from the model's outline onto the thin dark line that a bead's edge shows in the full-resolution image, within `refine_search_px` (10). A lone edge that jumps to a fold is rejected by a 9-sample outlier window. The record then holds the share of samples that were refined. It is off as shipped.

7.3 Millimetres

Pixels are not multiplied by a single factor. Both end points of each width sample are mapped through the calibrated measurement plane (chapter 8) and the distance between the mapped points is the width in millimetres. So perspective across the frame is accounted for, sample by sample.

If no calibration is saved, the fabric is `UNMEASURED`: pixel figures are shown, a warning is raised, and nothing is graded.

7.4 Grading

A segment passes when

```
target - tolerance  <=  segment average width (mm)  <=  target + tolerance
```

with `inspection.strip_width_mm` (4.0) and `inspection.strip_width_tolerance_mm` (1.0); both can be changed per size profile in the window. A fabric is `PASS` when every segment passes.

> **What this grading can and cannot see.** A segment is a tenth of the line (about 190 px, 25 mm). Its average hides a short narrow or wide spot, and a tolerance of +-1 mm is far wider than the 0.1-0.2 mm the user wants resolved. The statistical width test of chapter 16 was built for that finer question; it is a research tool and is not wired into the application.

7.5 What is saved for each fabric

`result.json` holds, per fabric: status, the polygon in frame pixels, confidence, the display box, and the measurement record with every segment's figures in pixels and millimetres. Preview images are JPEG; the numbers do not depend on them.

8 Calibration and millimetres

This chapter is a summary. The full account, with its proofs and acceptance test, is `docs/calibration/CALIBRATION_PIPELINE.md`; nothing here was researched anew.

8.1 From a board to a millimetre

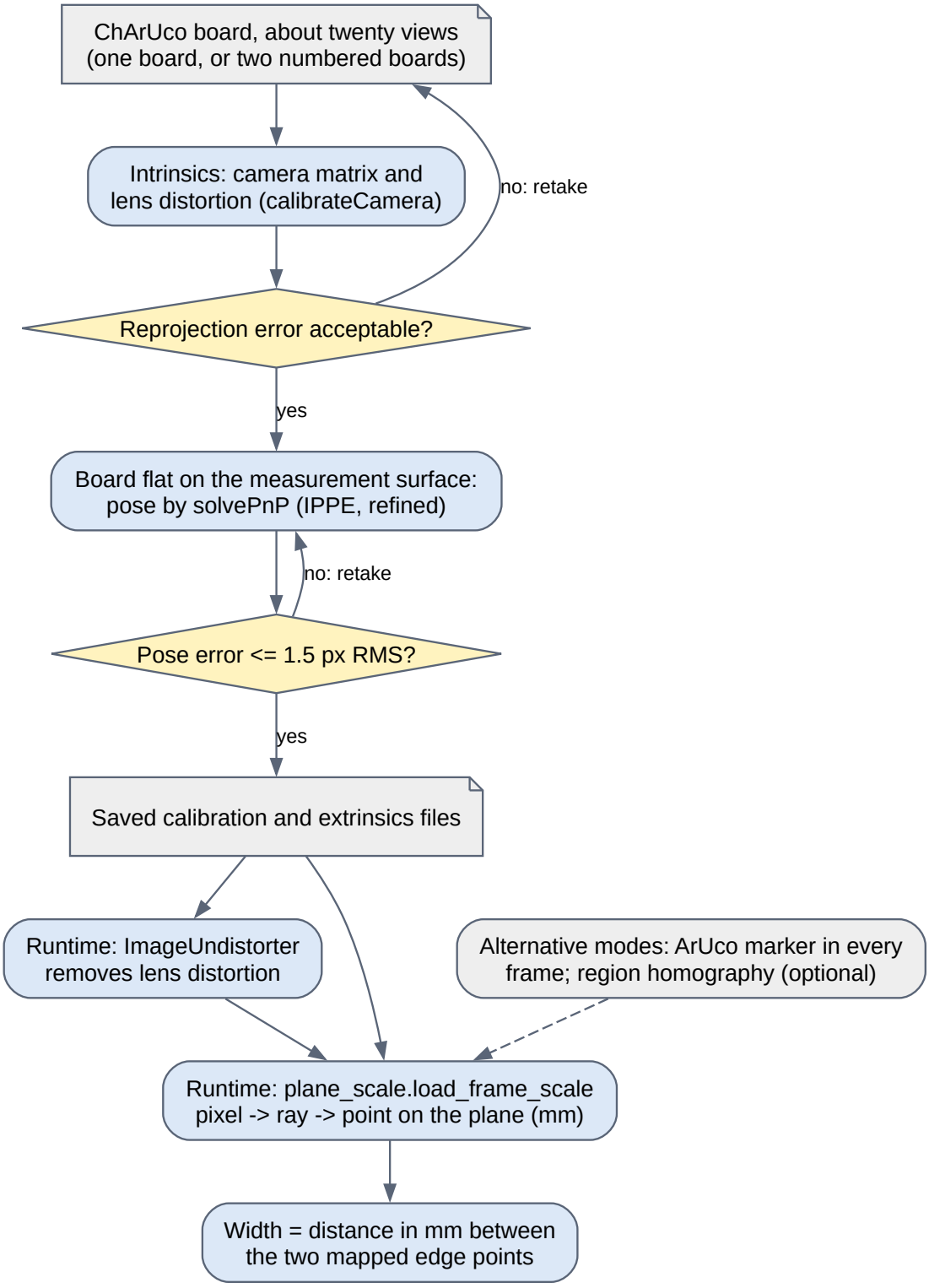


Figure 8.1 Calibration is done once; the two saved files are used on every capture.

Stage	What it gives	Accepted when
Intrinsics	Camera matrix and lens distortion, from about twenty views of a ChArUco board (one board, or two numbered boards)	reprojection error under the warning level (`camera_warning_rms_px`, 1.0)
Surface reference	The pose of the board lying flat on the measurement surface (`solvePnP`, IPPE start, refined)	at least 20 corners and at most 1.5 px RMS
Runtime	`ImageUndistorter` removes distortion; `plane_scale` turns a pixel into a ray and intersects it with the plane	files present and consistent with the frame size

Two other modes exist: an ArUco marker detected in every inspection frame, and an optional homography per crop region. Both are described in the calibration document.

8.2 Limits that matter for the glue line

- The scale is valid **on the calibrated plane**. Fabric that stands proud of the table, or a table that has been moved, changes the answer.
- Far from where the board lay, the result leans on the plane being flat there.
- Changing camera resolution, rotation or field of view needs a new calibration.

8.3 State on the development PC

> **The sample frames were taken with an uncalibrated camera.** Every millimetre figure in Part C therefore comes from a *fitted* scale that assumes good lines average 4 mm (chapter 16), not from this calibration. On those frames the same 4 mm strip reads 26 to 35 px depending on where it lies; about 7 % of a width reading is position, not glue. Calibrating the camera is the first of the next steps in chapter 22.

9 Configuration and saved data

9.1 Which module reads which settings

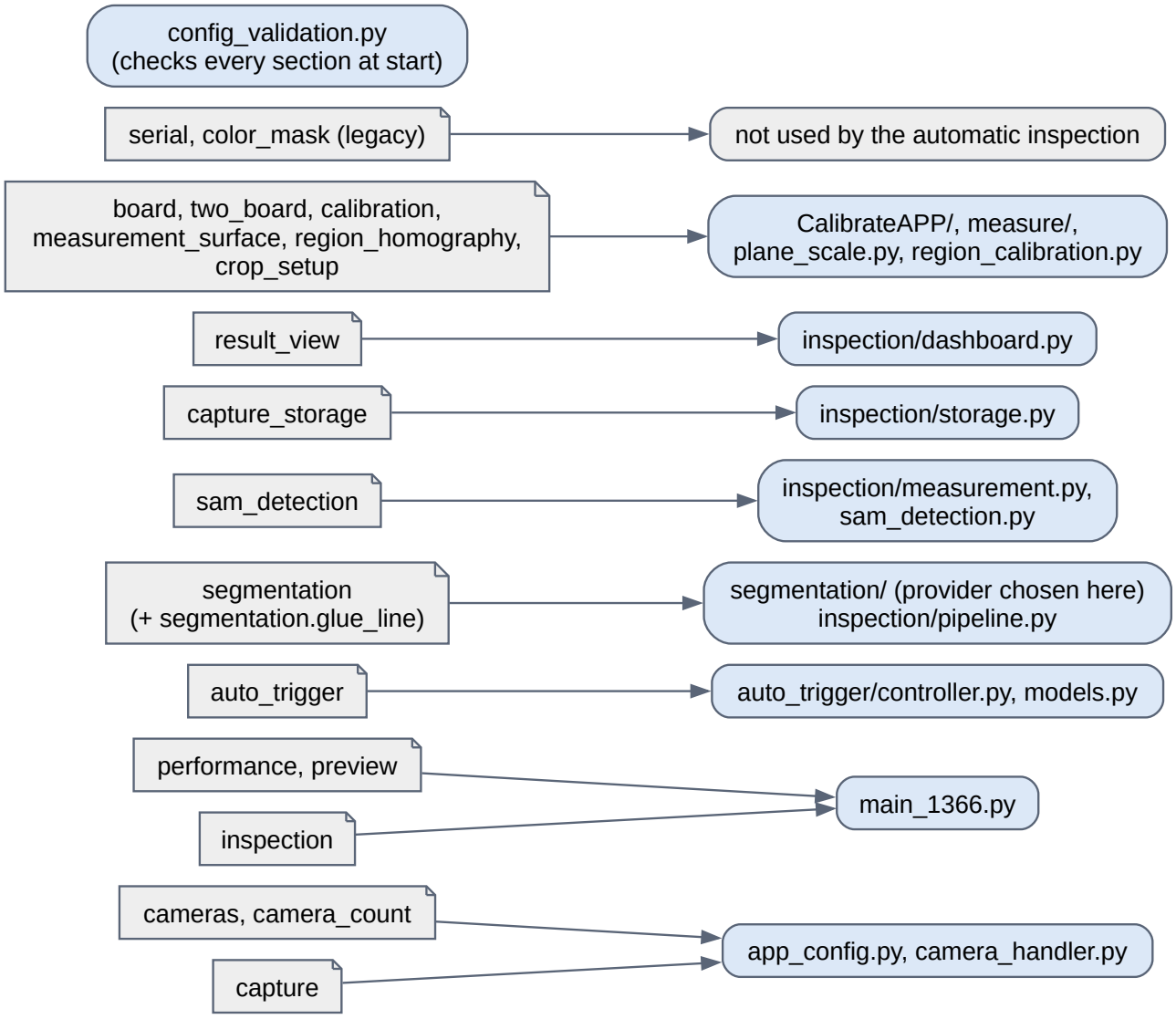


Figure 9.1 Sections of config.json and the modules that read them.

- `config.json` sits beside the code. At start, press **S** during the countdown for the settings form, or edit the file and restart.
- Invalid values are rejected at start by `config_validation.py`.
- **Do not put `null` in the file.** The settings form saves it back as the text "None".
- New keys are read with `.get` and a default, so an older `config.json` still loads.
- `.env` holds `ROBOFLOW_API_KEY` and is git-ignored.

9.2 The settings that shape an inspection

Setting	Shipped	Effect
`auto_trigger.hand_absence_seconds`	1.0	how long hands must stay away

Setting	Shipped	Effect
`auto_trigger.debounce_frames`	3	consistent frames before presence changes
`auto_trigger.detect_width`	320	width of the copy given to the hand detector
`auto_trigger.accepted_labels`	full, half fabric	which classifier labels go on to inspection
`auto_trigger.fabric_min_confidence`	0.5	classifier threshold
`segmentation.provider`	`workflow`	which provider (chapter 6)
`segmentation.roi`	whole frame	part of the frame sent to the workflow
`segmentation.fragment_policy`	`error`	stop on fragments, or keep the largest
`segmentation.min_confidence`	0.0	below this a strip is not graded
`segmentation.max_fabrics`	4	most fabrics in one capture
`sam_detection.strip_segments`	10	segments per line
`sam_detection.refine_edges`	false	edge refinement (chapter 7)
`inspection.strip_width_mm`	4.0	target width
`inspection.strip_width_tolerance_mm`	1.0	allowed +-
`inspection.result_display_seconds`	5.0	seconds per result tab
`capture_storage.max_sets`	1000	capture folders kept
`capture_storage.save_rejected_triggers`	false	also keep captures the fabric gate refused

Every key is described in `docs/inspection/CONFIGURATION.md` and `docs/inspection/AUTOMATIC_INSPECTION.md`.

9.3 A capture set on disk

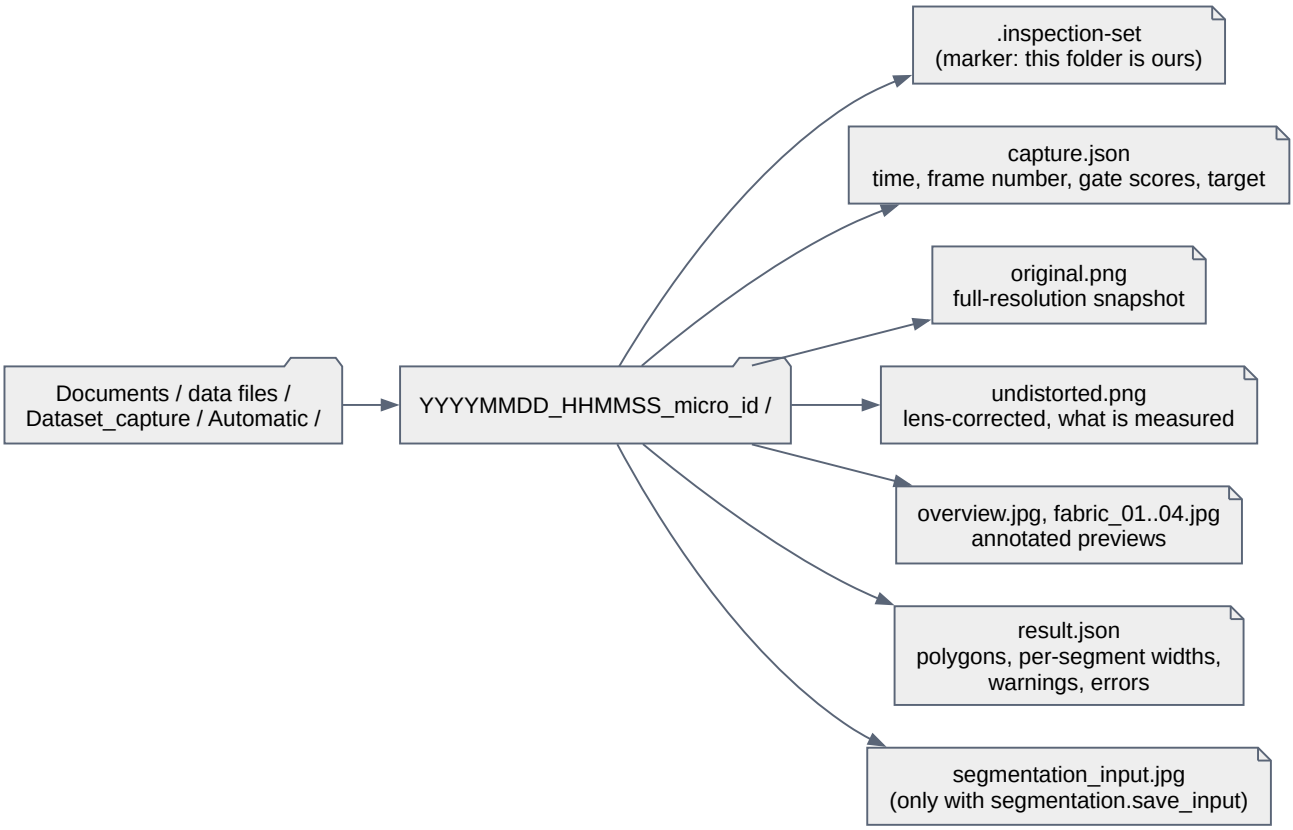


Figure 9.2 Files of one capture. The root is the user's Documents folder, not the code folder.

- `original.png` and `undistorted.png` are lossless and full resolution; the measurement was made on the undistorted one.
- Preview images are JPEG (`jpeg_quality` 82, longest edge `preview_max_edge` 1920).
- Polygons and numbers are in `result.json` in full-frame pixels, independent of previews.
- Retention deletes the oldest **completed** sets, whole folders at a time, and only folders that carry the `.inspection-set` marker. A set in progress is never deleted.
- A capture that was cancelled, rejected, superseded or failed is recorded with that status when it was saved at all.

PART C

Glue-line detection research (this branch)

10 The object and why it is hard

Part C describes research done with tools that are separate from the application. The full record, with every table and every method that failed, is `docs/glue-line/GLUE_LINE_FINDINGS.md`; this part is its structure and its pictures.

10.1 What the glue line looks like

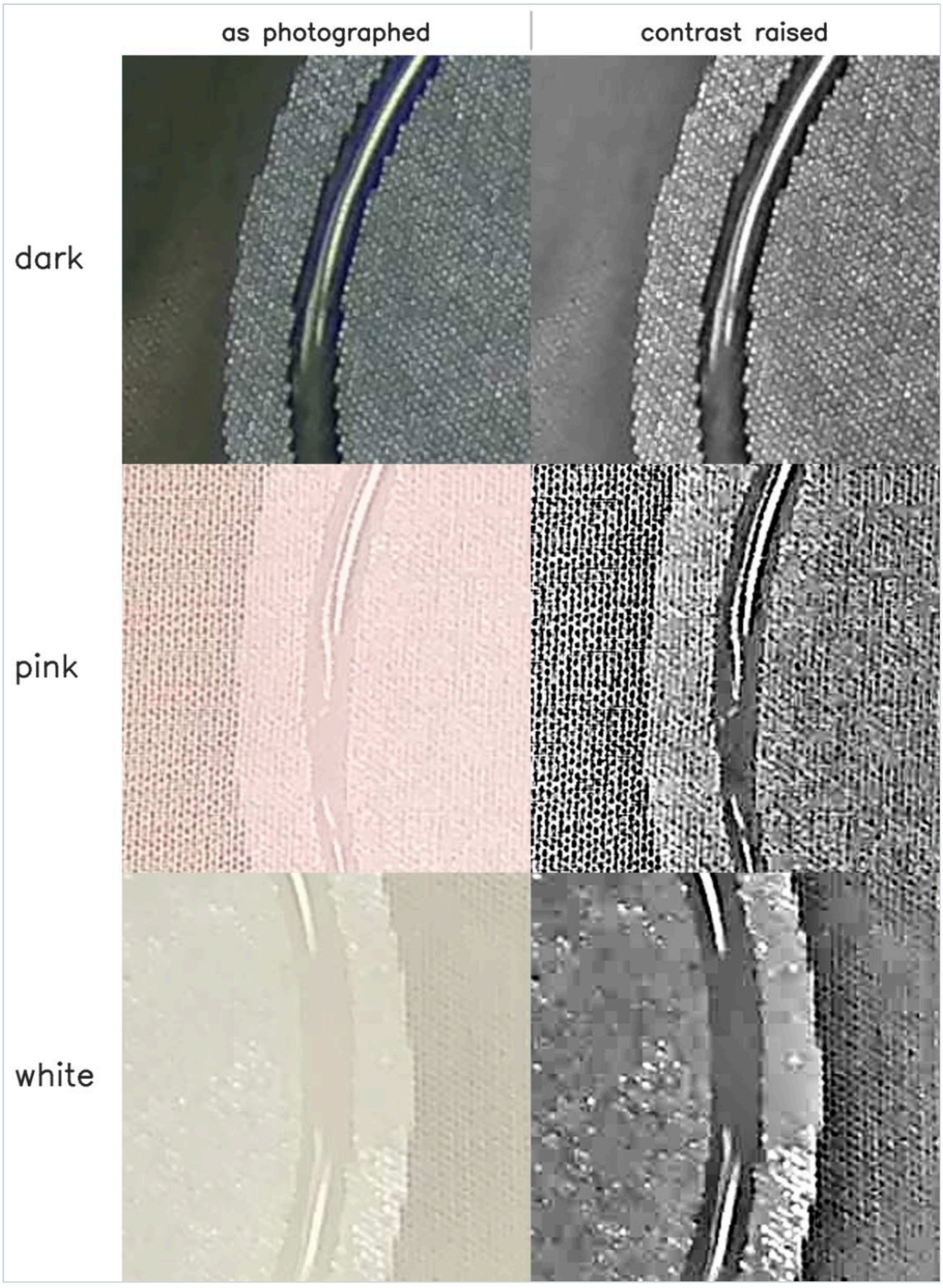


Figure 10.1 The middle of one glue line on each fabric colour, as photographed and with contrast raised. Each picture is about 260 px (34 mm) across.

- The glue is a thin glossy bead running along the curved edge of each panel strip. It makes a **band** about 30 px (4 mm) wide that is darker than the fabric, with a bright **core** that shows only where the light catches it.
- Every line has the same wave: two humps and a dip. Mirror images occur.

- A frame holds two to four lines, one per panel.
- Beside the bead on pale fabric lies a paler smooth band, 30-40 px wide. It is **not** the bead. Methods that outline 55-70 px have taken both together.

10.2 The signal, in numbers

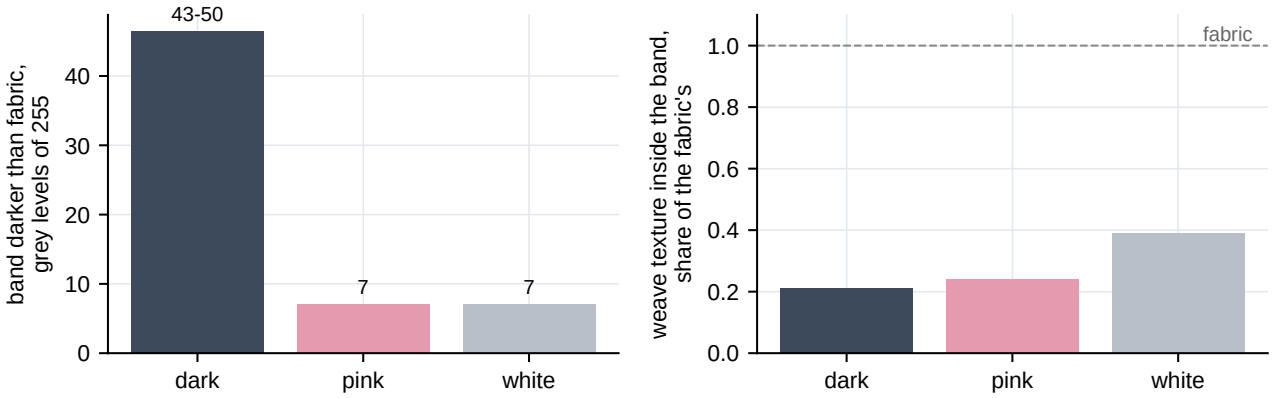


Figure 10.2 Left: how much darker the band is than the fabric. Right: weave texture inside the band as a share of the fabric's. Measured on 37 frames, 46 000 samples.

	Dark	Pink	White
Band darker than fabric, grey levels	43-50	7	7
Blur of the band's edge, px (sigma)	2.2	2.1-2.2	2.0-2.1
Shiny core over the band, grey levels	122	16	29
Weave texture inside the band, share of the fabric's	0.21	0.24	0.39
Width, px	31-32	30-31	33-34

Two consequences run through everything that follows.

1. **On pale fabric the signal is a seventh of what it is on dark fabric**, and it is of the same size as the weave's own pattern. Nothing can be read from single pixels there; the picture has to be averaged *along the line*.
2. **Glue fills the weave.** Where the band is barely darker than the fabric, it is still two to five times smoother. Used alone the texture cannot place an edge, but as an input to a trained edge corrector it helps (chapter 14).

10.3 What makes it hard

Difficulty	Where	What it causes
Low contrast	pink, white	segmenters outline the wrong, wider band or spill sideways
Shine comes and goes along one line	all colours	an edge finder steps sideways where the core lies along one edge
Soft frames	white frames of other image sets (sharpness 1-6 against 90-600)	SAM marks half the lines doubtful
Mesh, cups, folds, seams, cardboard	everywhere outside the panel strip	false lines unless the search is confined to the panel region
No calibration	all sample frames	the same strip reads 26-35 px by position
No defective samples	all	no method can be shown to catch a real defect

10.4 Images used

Set	Path (git-ignored)	Content
M1 captures	`enhancement_images/M1_Masks/captures`	74 frames, 4608 x 3456: 34 dark, 34 pink, 6 white; 257 lines
10_05	`enhancement_images/10_05`	4000 x 3000: navy, soft white, another part
Real captures	`enhancement_images/real_captures`	6 webcam frames

Almost every number in Part C is from the M1 captures. Fourteen bursts of 3-5 shots of the same layout inside that set serve as repeat shots.

11 How the flow evolved

11.1 Versions and the verdict on each

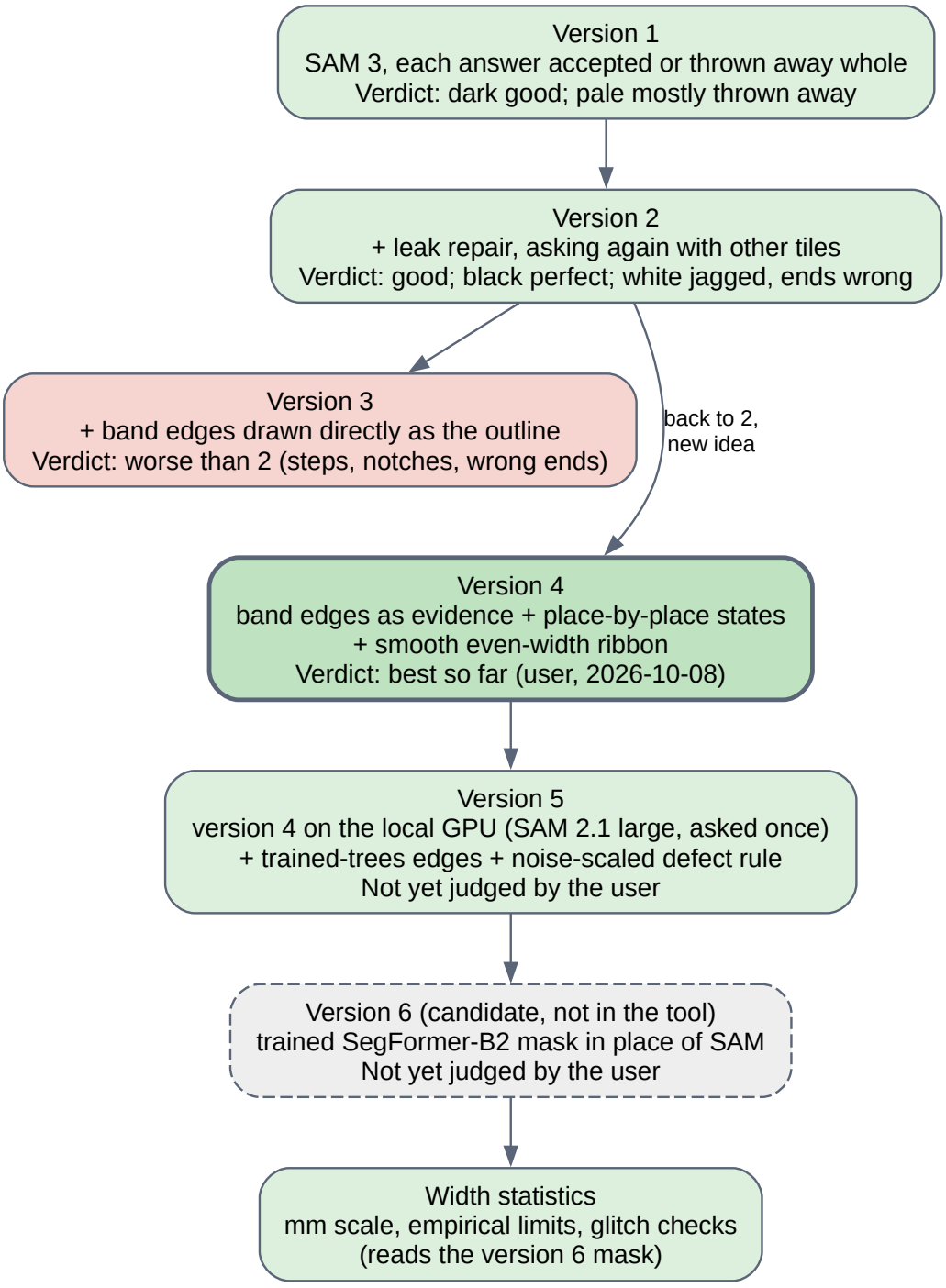


Figure 11.1 Versions of the detection flow. The thick box is the one the user judged best.

Version	Outline comes from	Verdict
1	SAM 3, each answer accepted or thrown away whole	dark good; pale mostly thrown away
2	SAM 3 + leak repair + asking again	good; black perfect; white jagged, ends too short or too long
3	version 2 + band edges drawn directly as the outline	**worse** than 2: steps, notches, wrong ends
4	version 2 + band edges as evidence + states + smooth ribbon	**best so far** (user, 2026-10-08)
5	version 4 on the local GPU + trained-trees edges + noise-scaled defect rule	not yet judged by the user
6 (candidate)	trained SegFormer-B2 mask in place of SAM, same trees and ribbon	not built into the tool; not judged

11.2 The same line through versions 2 to 5

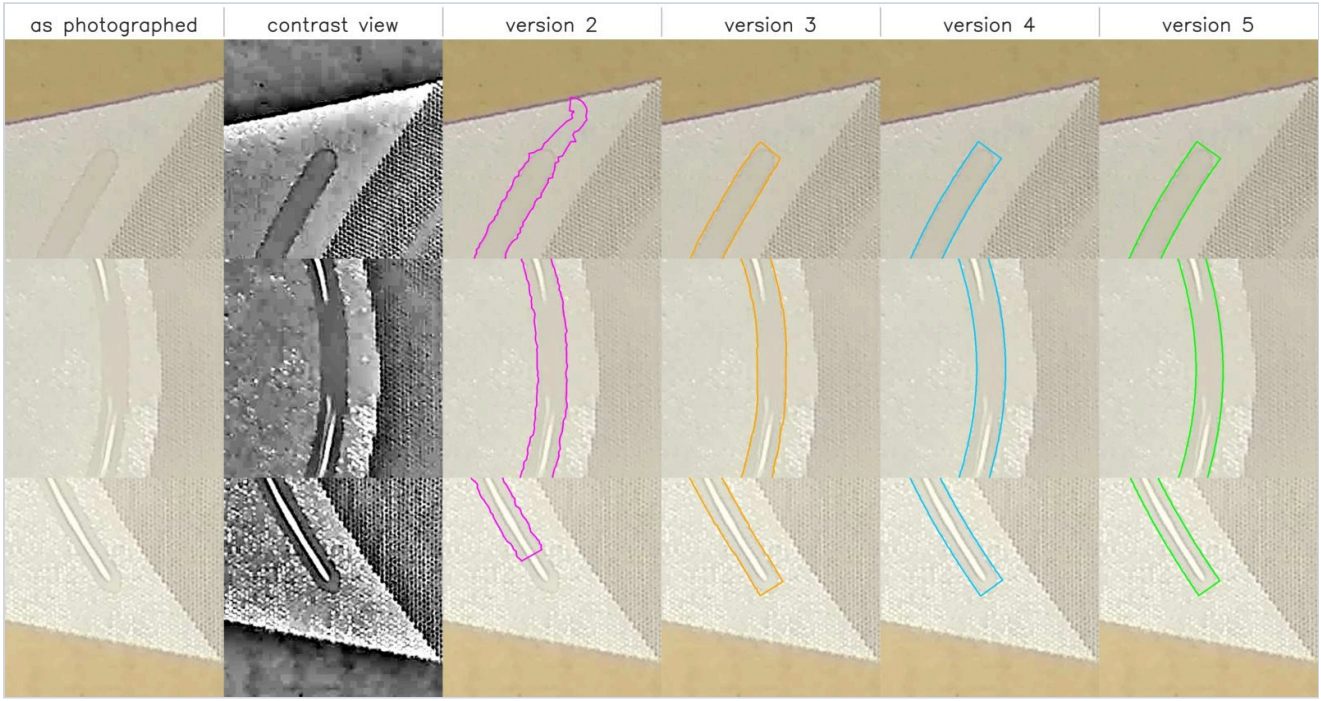


Figure 11.2 One white line: start, middle and end. Version 2 is jagged and runs past the tips; version 3 has straight but notched edges; versions 4 and 5 are smooth ribbons that end where the band ends.

11.3 What each step taught

Step	Lesson
1 to 2	SAM spills in places on pale fabric, and in *different* places each time it is asked. Keep its steady edges and ask again, instead of filtering the picture.
2 to 3	A classical edge finder is not a better outline. It follows the core where the core lies along one edge.
3 to 4	Neither source is right everywhere. Treat both as evidence, weight them by what the picture is like at each place, and fit the shape the glue is known to have: a smooth curve of even width.
4 to 5	Measure error against a known truth before changing anything. The cloud is the slow part; a local SAM asked once gives the same ribbon.
5 to 6	A network trained on the flow's own best outlines reads soft frames that SAM cannot.

11.4 What was tried and does not work

Measured, and recorded so that it is not tried again.

- **Any filter in front of SAM**: CLAHE, flatten lighting, bead enhance, contrast gain, denoise, unsharp mask, negative-kernel sharpen, negative, quantise, shine compress, dimming outside the ROI, smoothing along the line, and combinations. Equal at best; a contrast boost makes SAM outline the wider band.
- **Enlarging tiles** two or three times: the outline fragments.
- **A whole-image filter before the local detector**: 34 of 40 panels with or without.
- **Drawing band edges as the outline** (version 3).
- **Reporting a width deviation from one source alone**: band edges alone flag 40-87 % of good lines.
- **Sub-pixel peak fitting, phase congruency, the texture channel alone, averaging several shots**: no gain in edge accuracy.
- **MobileSAM and FastSAM** on pale fabric.
- **CLAHE for the trained segmenter**: no gain on sharp frames, worse on soft ones.

12 The detection flow in use, stage by stage

12.1 The whole flow

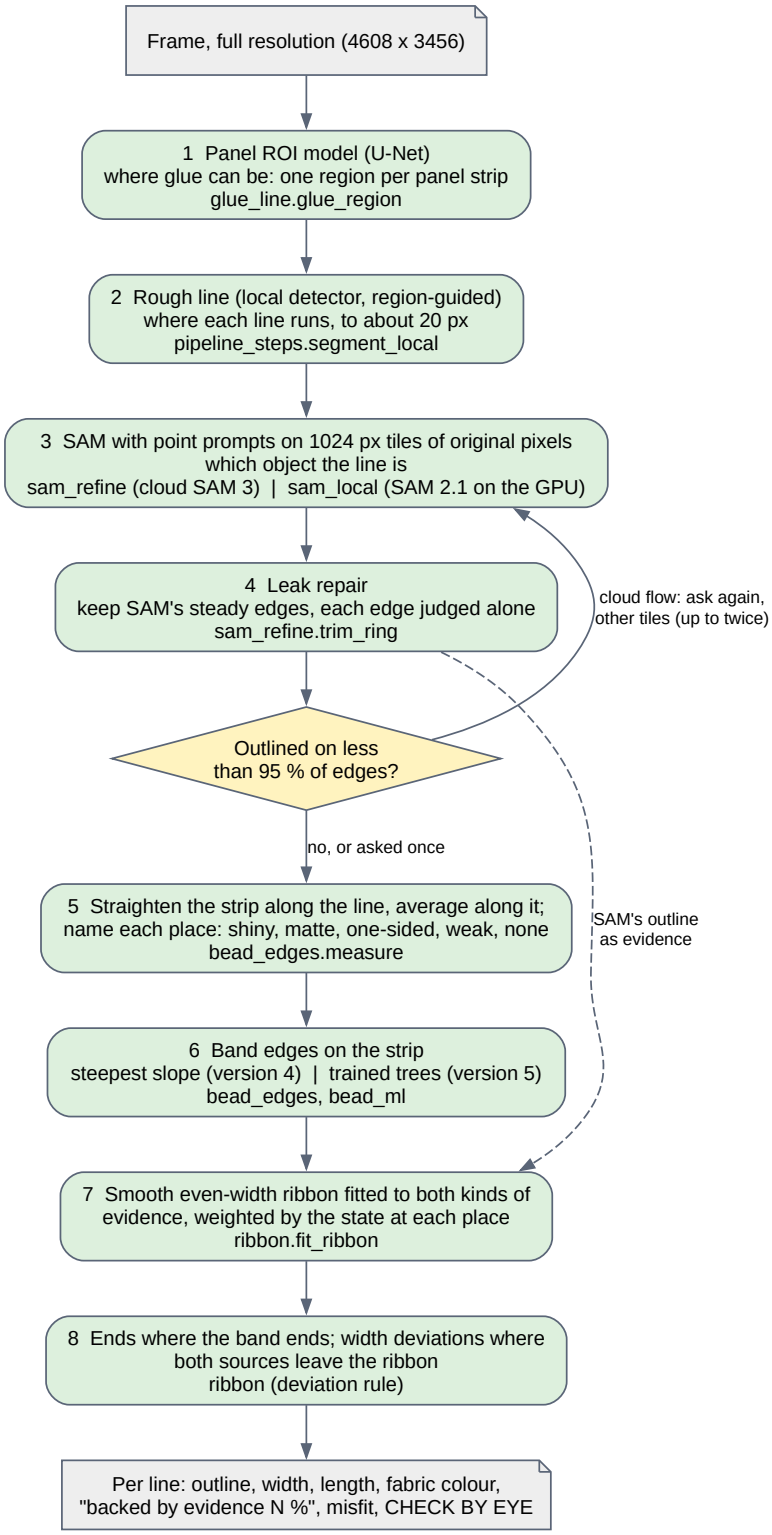


Figure 12.1 The flow of versions 4 and 5. Each stage does one job, and no stage is trusted for what it is bad at.

In the Pipeline Lab this is: ROI *Panel model*, no pre-processing steps, *SAM - points placed automatically*, *Smooth ribbon from the true shape* ticked; for version 5 also *SAM runs on: this PC's GPU*, *Ask SAM once per line* and *Edges placed by: trees*.

12.2 One frame through the main stages

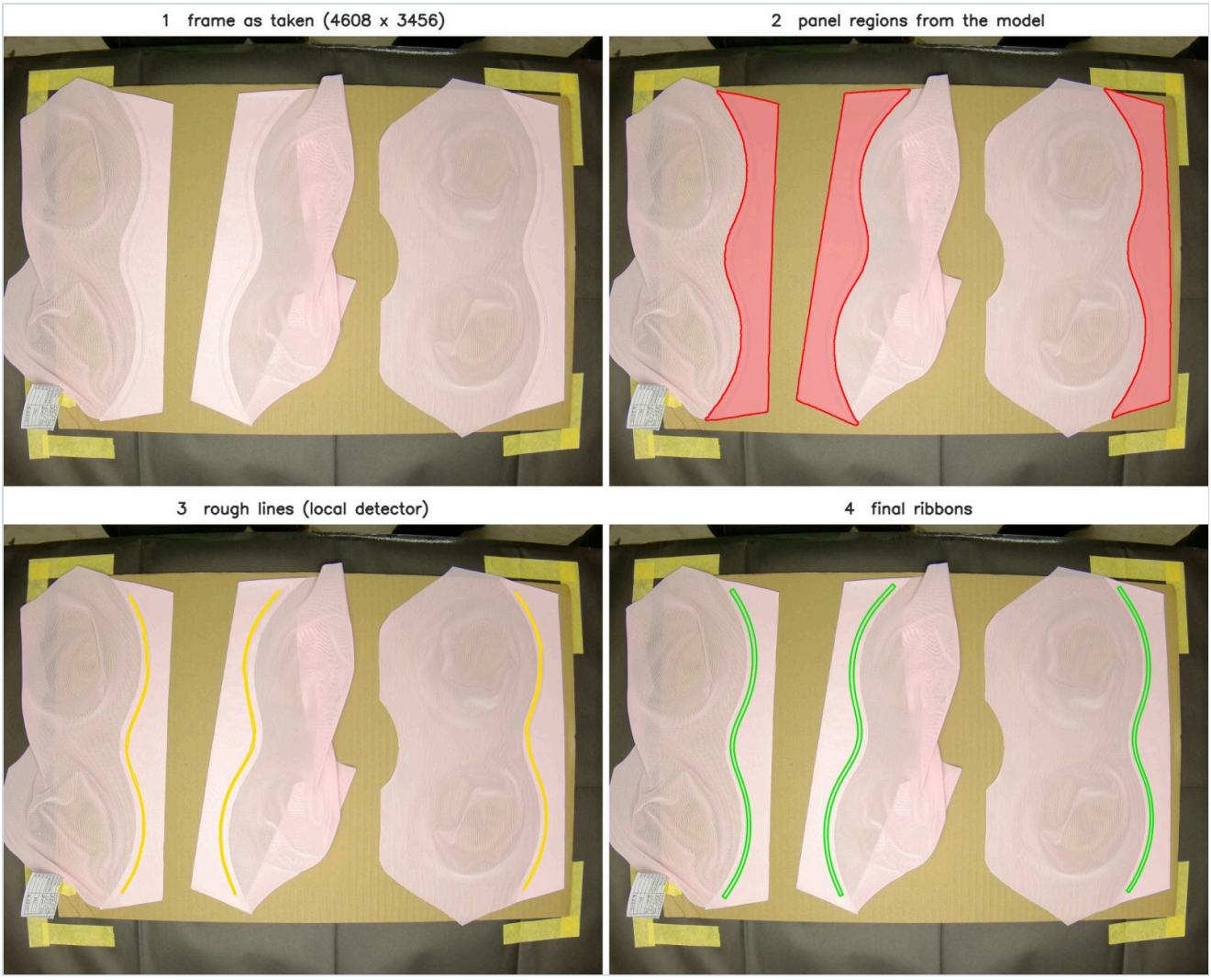


Figure 12.2 A pink frame of three panels. Stage 2 marks where glue can be; stage 3 is only a course; stage 4 is the final outline of each line.

12.3 Stage 1: panel region

Aspect	Detail
Job	Say where glue can be: one region per panel strip
How	U-Net with a ResNet-34 encoder, input 1152 px wide, mirrored second pass; cardboard removed from the mask
Code	`glue_line.glue_region`, `pipeline_steps.model_roi`; scripts in `model_for_segment_bra_roi places/`
Time	2.1 s on the CPU (ONNX); 0.25 s on the GPU (PyTorch checkpoint; masks agree, IoU 0.99998)
Fails when	a panel is missed: then that line is never looked for
Why	everything outside the region (mesh, cups, cardboard) is the source of false lines

12.4 Stage 2: rough line

Aspect	Detail
Job	Say roughly where each line runs, on the original pixels
How	The local detector in region-guided mode: one line inside a corridor along the curved side of each region
Code	`segmentation/glue_line.py`, `pipeline_steps.segment_local`
Time	8-11 s with whole-frame filters; 0.7-0.9 s with `guided_skip_frame_filters` (same lines, 44 of 44)
Needs to be	within about 20 px; it is a course, not an outline

12.5 Stage 3: SAM with point prompts

Aspect	Detail
Job	Say which object the line is
How	Points every 150 px along the rough line, on 1024 x 1024 tiles of **original** pixels. A far too wide answer is asked again with "not this" points 45 px to each side
Code	cloud: `segmentation/sam_refine.py` (SAM 3, Roboflow); local: `segmentation/sam_local.py` (SAM 2.1, half precision, a tile encoded once and asked many times)
Time	cloud 9-33 s per frame; local 3.2-4.6 s
Fails when	pale fabric: it spills into the band or mesh beside the bead over part of the line
Rule	never a reduced frame, never a filtered one

12.6 Stage 4: leak repair

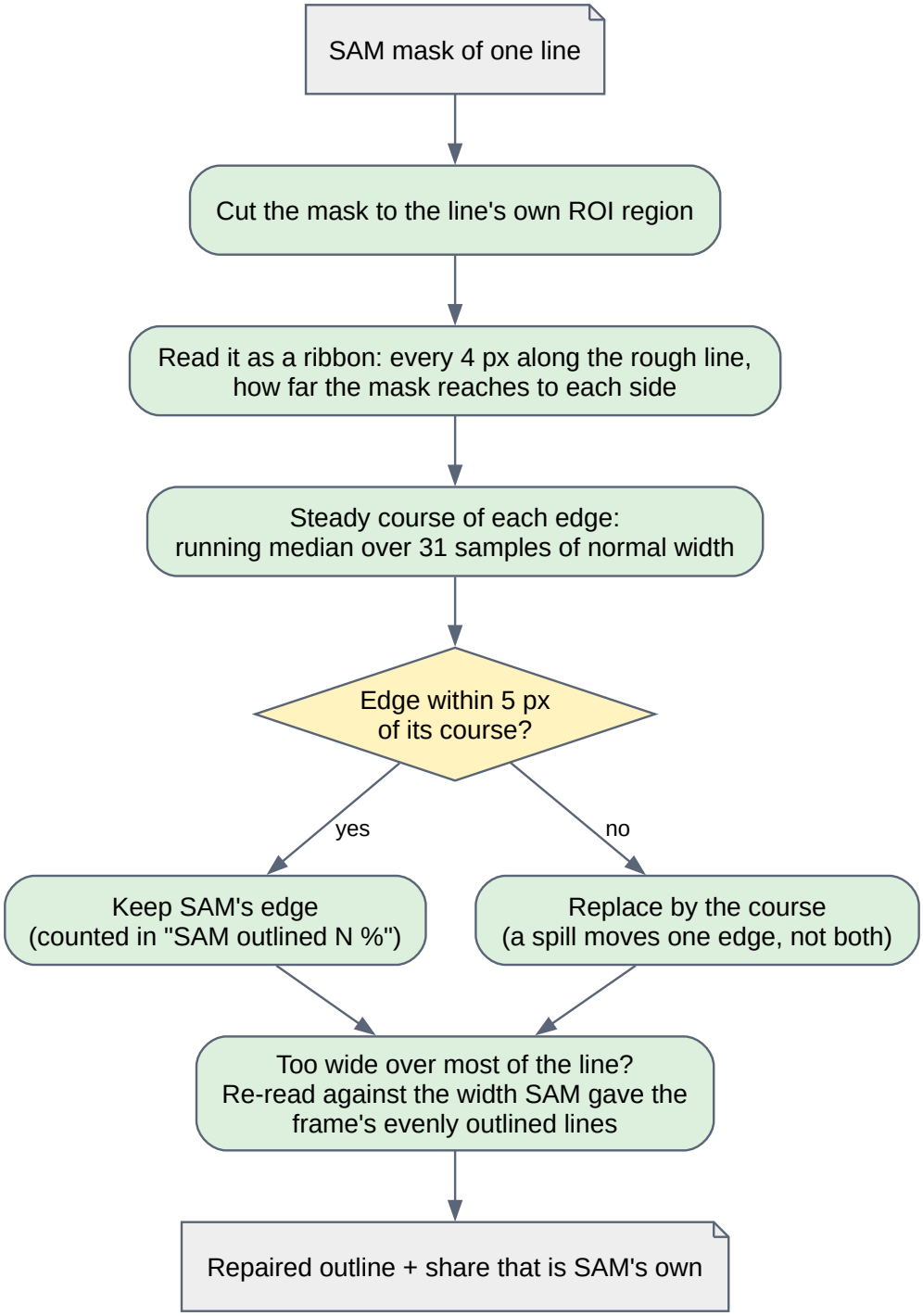


Figure 12.3 Leak repair. Each edge is judged alone because a spill moves one edge and leaves the other.

In the cloud flow a line outlined on less than 95 % of its edges is put to SAM again, up to twice, with other tile strides and point spacings (560/110 and 1000/200 px); the steady parts of the answers are combined. In the local flow each line is asked once, because the band edges of stage 6 cover what SAM missed.

12.7 Stages 5 to 8: states, band edges, ribbon

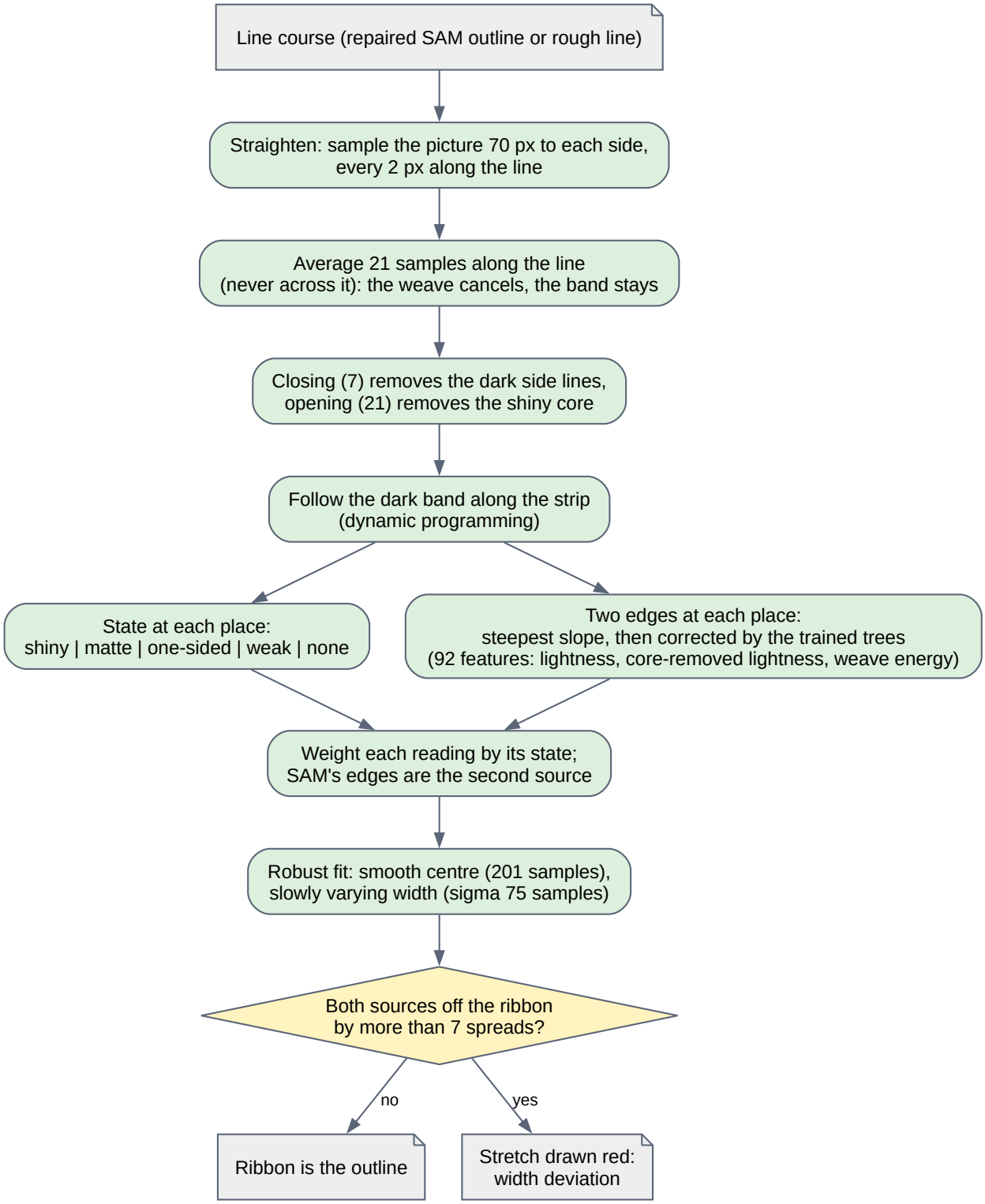


Figure 12.4 From the line's course to the ribbon. The states decide how much each reading counts.

State	What the picture shows there	Weight of the band edges	Weight of SAM's edges
shiny	band with a bright core in the middle	1.0	1.0
matte	band without a core	1.0	1.0
weak	the band is faint	0.4	0.6
one-sided	the core lies along one edge, which pulls the band edge inwards	0.15	1.0
none	no band	0	0.3

The weights are `STATE_WEIGHT` in `segmentation/ribbon.py`.

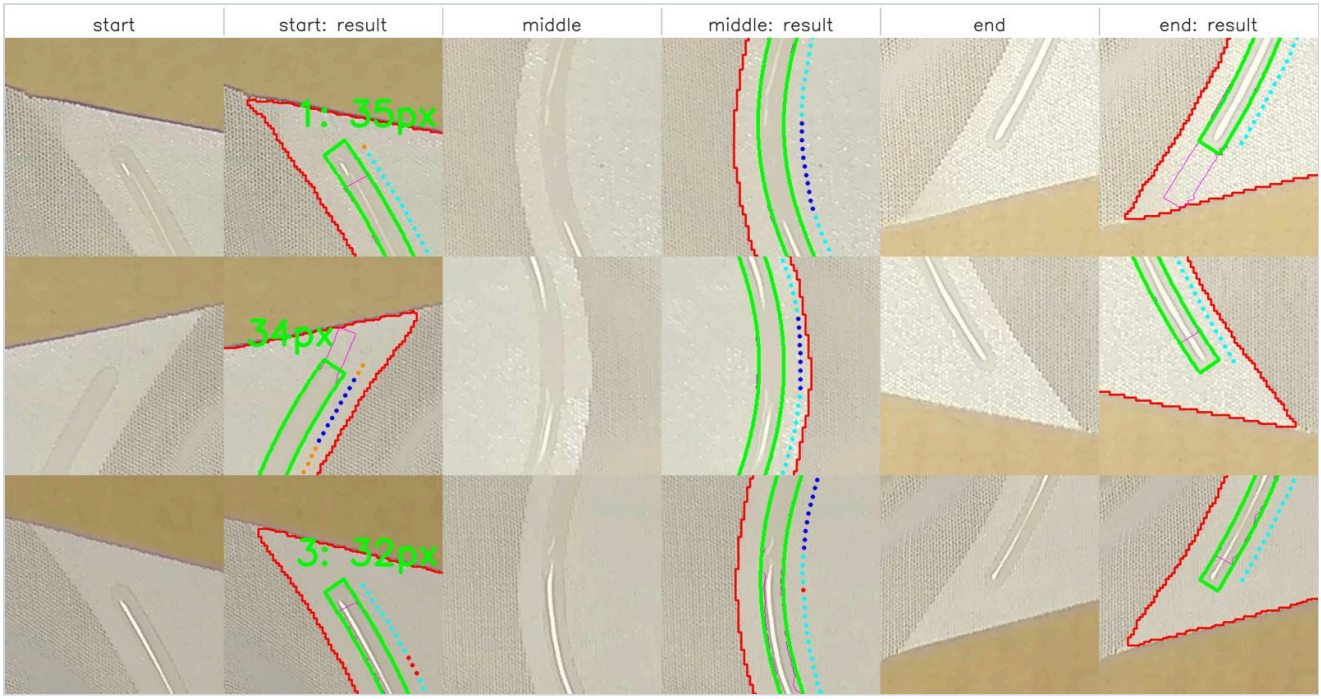


Figure 12.5 A white frame from the version 4 run: three lines at start, middle and end. Green: the final ribbon. Magenta, thin: SAM's own outline. Red: border of the panel region. Dots: state at each place (cyan shiny, blue matte, red one-sided, orange weak).

The ribbon fit takes the centre line as a smooth curve and the width as slowly varying, because that is what the glue is. It reports, per line, the share of its length **backed by evidence** and the **misfit** in pixels, and marks the line CHECK BY EYE when either is poor. A stretch where *both* SAM's edges and the band's edges are steadily wider or narrower than the ribbon, by more than 7 spreads of the line's own noise, is drawn red as a width deviation.

12.8 What the flow gives on the sample set

Version 5, 74 frames	Dark	Pink	White
Lines marked "ok"	126 of 126	113 of 113	18 of 18
Width, median (range), px	32.0 (29.6-33.2)	29.2 (24.8-34.8)	32.2 (26.4-34.4)
Width deviations reported	0	9	1
Seconds per frame	5.7	6.4	4.7

Since the samples have no defects, the ten deviations are false alarms; with the two-source rule at threshold 7 the count over all 257 lines is 1 (chapter 14).

13 Speed: the local GPU flow

13.1 Where the time goes

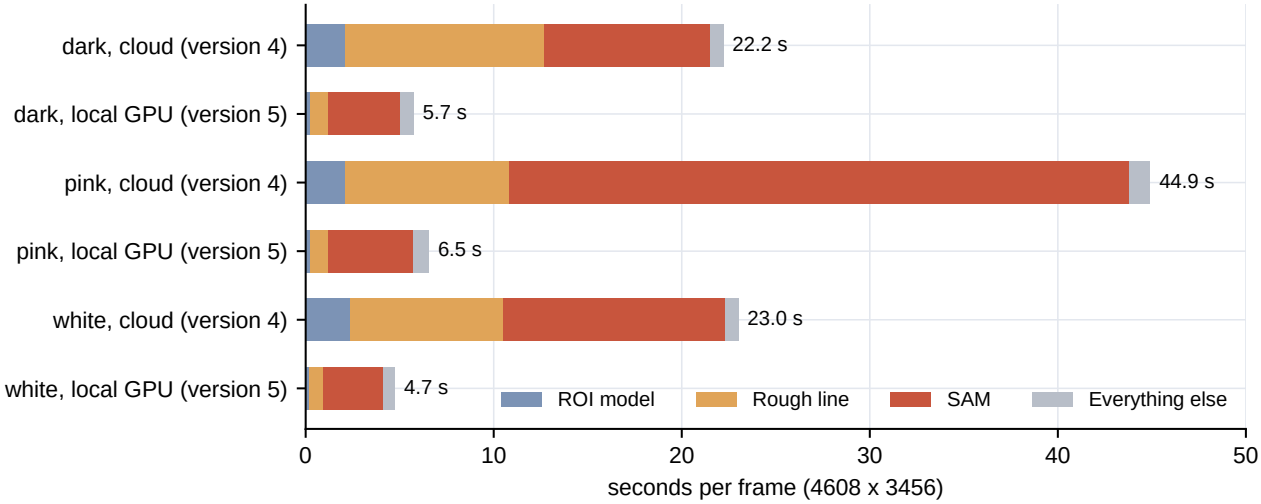


Figure 13.1 Seconds per frame by stage, cloud flow against local GPU flow, mean of three frames per colour.

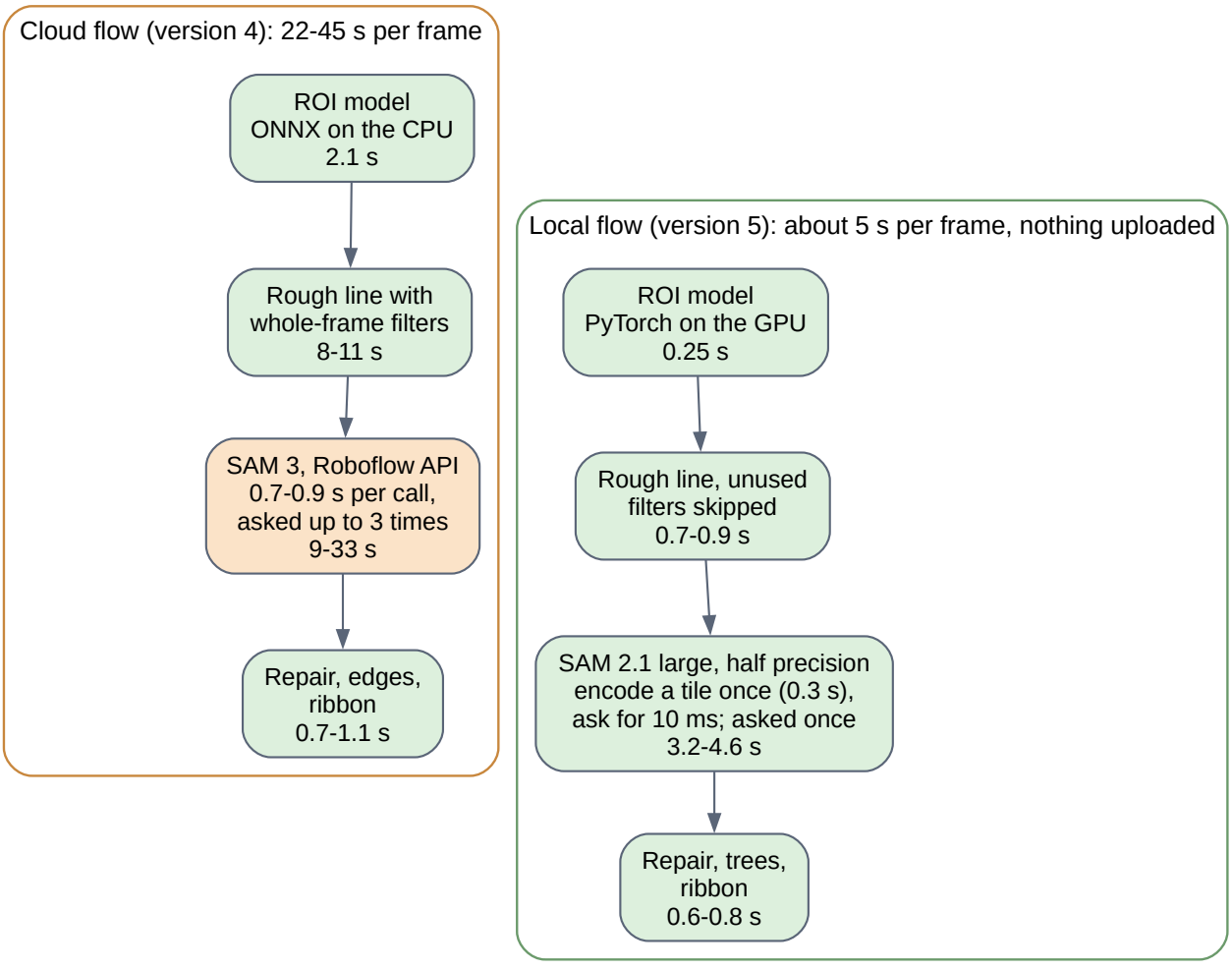


Figure 13.2 The two flows side by side. The stages are the same; where they run and how often SAM is asked differ.

Flow	Seconds per frame: dark / pink / white	Uploads	GPU memory	Against cloud version 4
Cloud version 4	22.1 / 44.9 / 23.0	every tile	-	reference
Local SAM 2.1 large, asked once, ribbon	**5.7 / 6.5 / 4.8**	none	1.0 GB	centre line 0.1-0.5 px away; width 0.4-1.7 px narrower; 257 of 257 lines "ok"
Local SAM 2.1 tiny	3.1 / 3.3 / 2.6	none	0.6 GB	4 pink lines "check"
No SAM: rough line + band edges + ribbon	1.2 / 1.2 / 1.0	none	0.5 GB	19 pink lines "check"; 2 lines over 3 px off

Measured on the development PC: Intel i5-13420H, RTX 3050 Laptop GPU with 6 GB, PyTorch 2.11 with CUDA. The first run of a session takes 11-16 s while models load.

13.2 What made the difference

Change	Effect
SAM on the PC's own GPU in half precision	a tile is encoded in 0.3 s and can then be asked again for about 10 ms; a cloud call takes 0.7-0.9 s
Each line asked once	the band edges cover what SAM missed, so the repeat askings add nothing once the ribbon is on
Whole-frame filters skipped in the rough line	90 % of that stage's time went into filters the region-guided search never read
Region model on the GPU through its PyTorch checkpoint	2.1 s to 0.25 s

Half precision is the one setting that matters for SAM: full precision is 5.8 times slower and needs 3 GB. Batching tiles gains nothing.

13.3 Proof that it is offline

One frame was run with the network removed (`unshare -n`), `HF_HUB_OFFLINE=1`, no API key in the environment and the cloud client replaced by one that raises an error. It completed: three lines, widths 33.6, 32.4 and 30.0 px.

13.4 Not done

- Nothing was timed on the Windows inspection PC.
- `torch.compile`, TensorRT and ONNX export of SAM were not tried.
- Local SAM 3 was not run: its weights are gated and need a Hugging Face login.
- The application's own cycle (`docs/inspection/PERFORMANCE.md`) was not re-timed.

14 Accuracy: the truth bench

Until this work every result was judged against another method or by eye. "Half a pixel from version 4" says two methods agree, not that either is right. The bench measures error against a known truth and keeps a method only if the error falls.

14.1 How the bench works

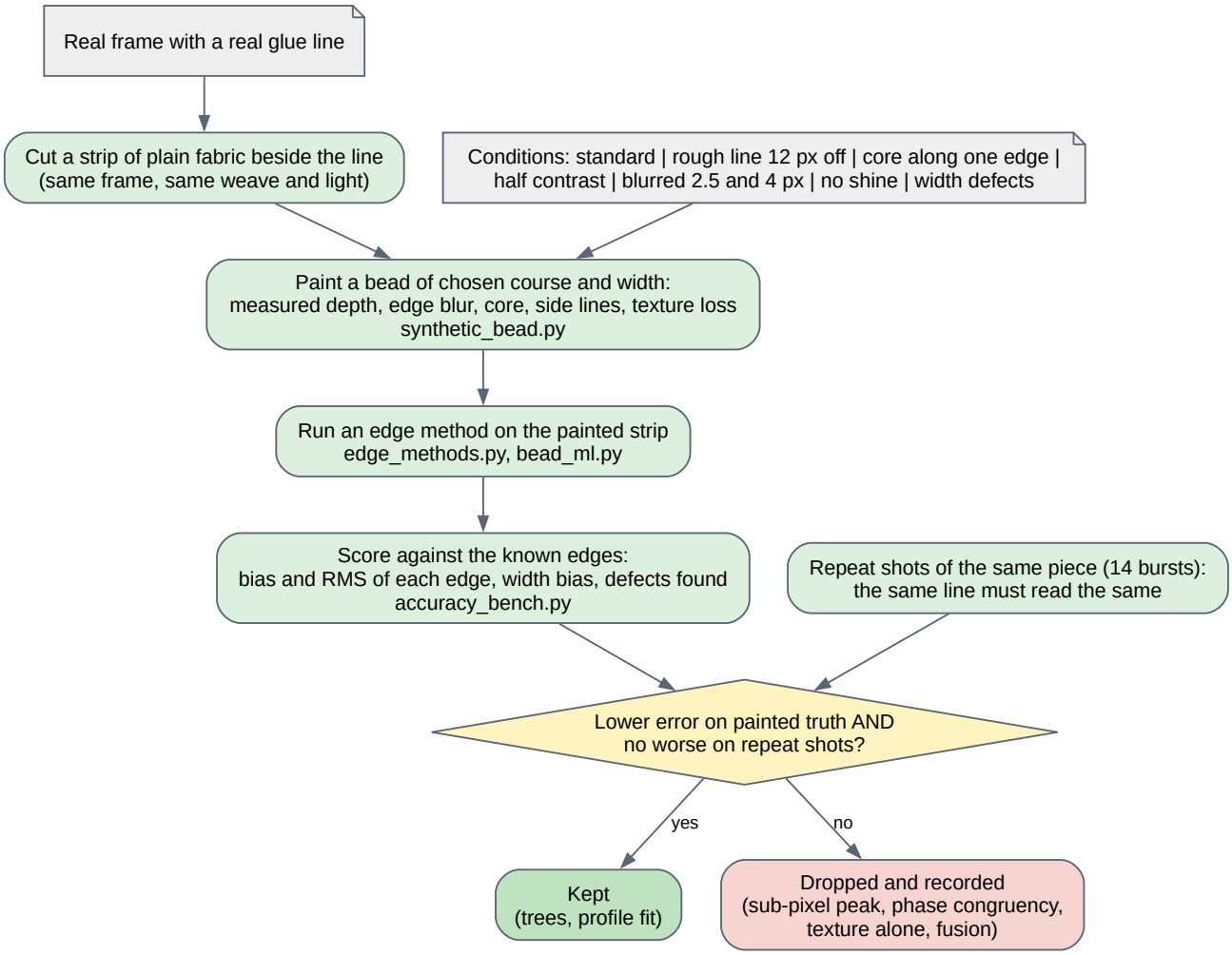


Figure 14.1 Painted truth and repeat shots, and the rule for keeping a method.

- **Painted glue lines on real fabric.** A strip of plain fabric is cut from the same frame beside a real line, and a bead is painted into it with the measured depth, edge blur, core, dark side lines and loss of texture, along a chosen course with a chosen width. Its edges are therefore known exactly.
- **Repeat shots.** Fourteen bursts of 3-5 shots of the same layout (51 lines). The same line must read the same in each.
- A painted line is a model of the glue line. Passing the bench is necessary, not sufficient.

14.2 Edge methods compared

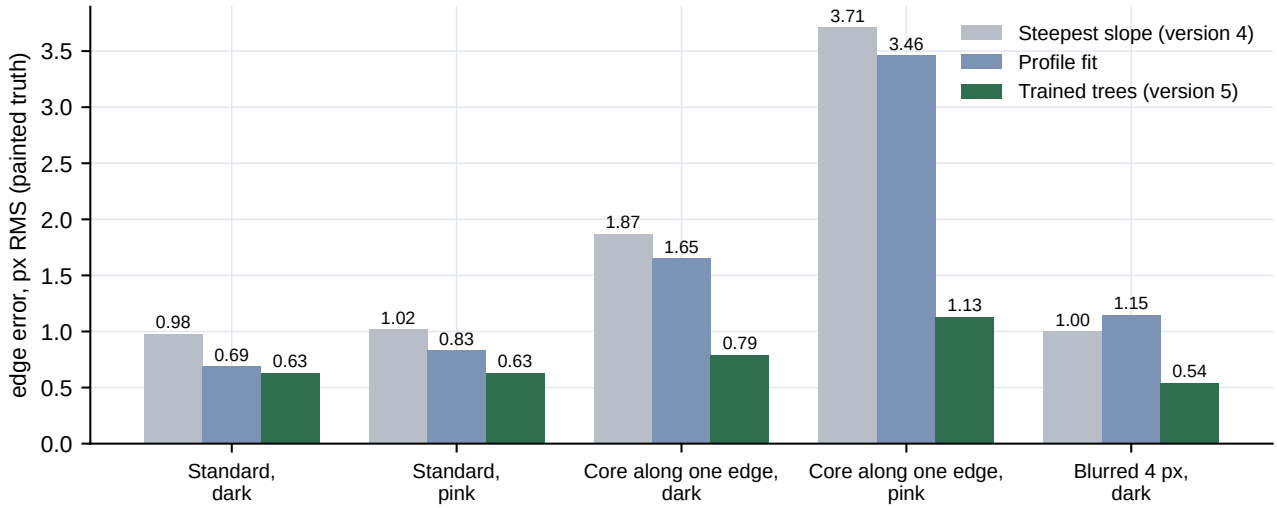


Figure 14.2 Edge error against painted truth on frames held out from training. Lower is better.

Method	Result	Verdict
Steepest slope, whole pixels	about 1 px RMS; pulled in by 1.3 px on pink where the core lies along one edge	the baseline (version 4)
Same, sub-pixel by parabola	no gain; worse on white	dropped: noise, not pixel size, is the limit
Steepest slope at a coarser scale	small gain; worse when blurred	dropped
Phase congruency	equal to the baseline	dropped
Weave-texture channel alone	9-94 px	dropped: the texture edge cannot be placed
Lightness and texture fused	helps dark only	dropped
Parametric profile fit	better (0.7-0.9 px); reads wide when blurred	kept as an option
Trained trees	**0.63 px**; no bias from a one-sided core or from blur	**kept: version 5**

Trees are gradient-boosted decision trees (LightGBM, 600 trees, 92 features) that correct the steepest-slope edge from the profile around it in three channels: lightness, lightness with the core removed, and weave energy. They were trained on painted lines from 52 frames and tested on 22 frames never seen. The model file is `~/.cache/flod_sam/bead_edge_trees.txt`, written by `tools/train_bead_ml.py`.

14.3 Precision on real repeat shots

Edge method	Width of the same line between shots, SD px (dark / pink)	Width noise along a line, px
Steepest slope	0.10 / 0.13	0.73 / 1.26
Profile fit	0.08 / 0.20	0.83 / 1.51
Trees	**0.07 / 0.14**	**0.64 / 1.11**

The whole-line width repeats to about a tenth of a pixel. Averaging several shots gains almost nothing: what limits a reading is the fixed pattern of the weave, not camera noise.

14.4 Width defects and false alarms

On the bench the first defect rule found 2 of 192 painted defects. The rule now compares the line's width with its own running median and scales the threshold by the line's own noise; it finds 85 of 192, and reliably only a defect of about 8 px over 60 px (about 1 mm over 8 mm).

Then the user stated that the supplied strips have no defects. Every deviation reported on them is therefore false, which gives a real false-alarm count:

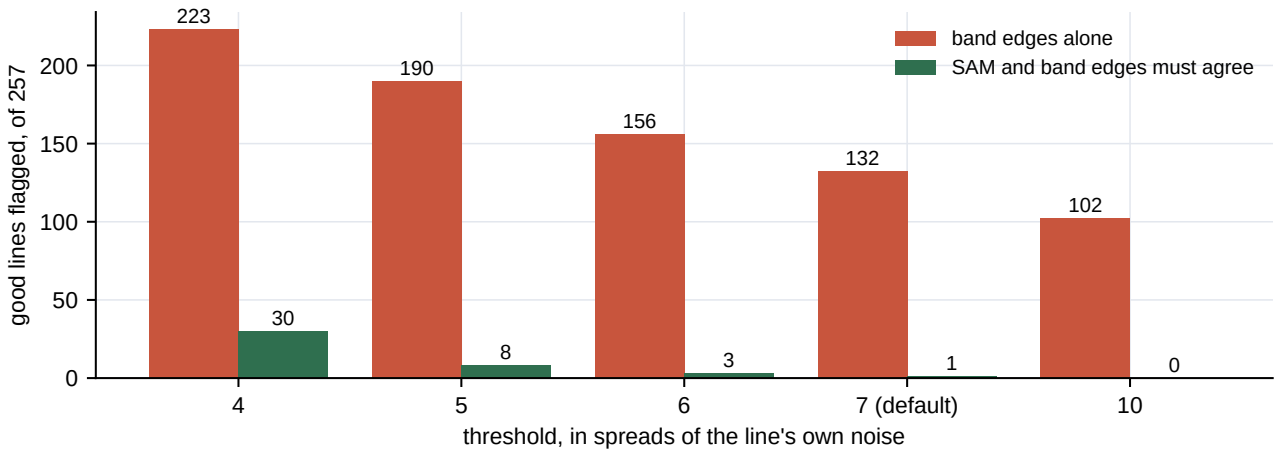


Figure 14.3 Good lines falsely flagged, of 257, by threshold. Band edges alone are unusable at any threshold.

- **Only the rule that requires SAM's edges and the band's edges to agree is usable.** At threshold 7 it flags 1 good line of 257.
- Deviations that recurred across repeat shots were steady artefacts of the picture, not real changes. Recurrence does not confirm a defect.

14.5 In millimetres

With about 7.6 px per mm (uncalibrated): edge error of 0.5-0.7 px RMS is about 0.07-0.09 mm; the width of a whole line repeats between shots to 0.01-0.02 mm.

> **What this does not establish.** How many *real* defects the two-source rule finds is unknown: there is no defective sample, and the painted bench understates real disturbance. This is the largest open question of the whole project.

15 The trained segmenter (SegFormer and CLAHE)

15.1 Training and use

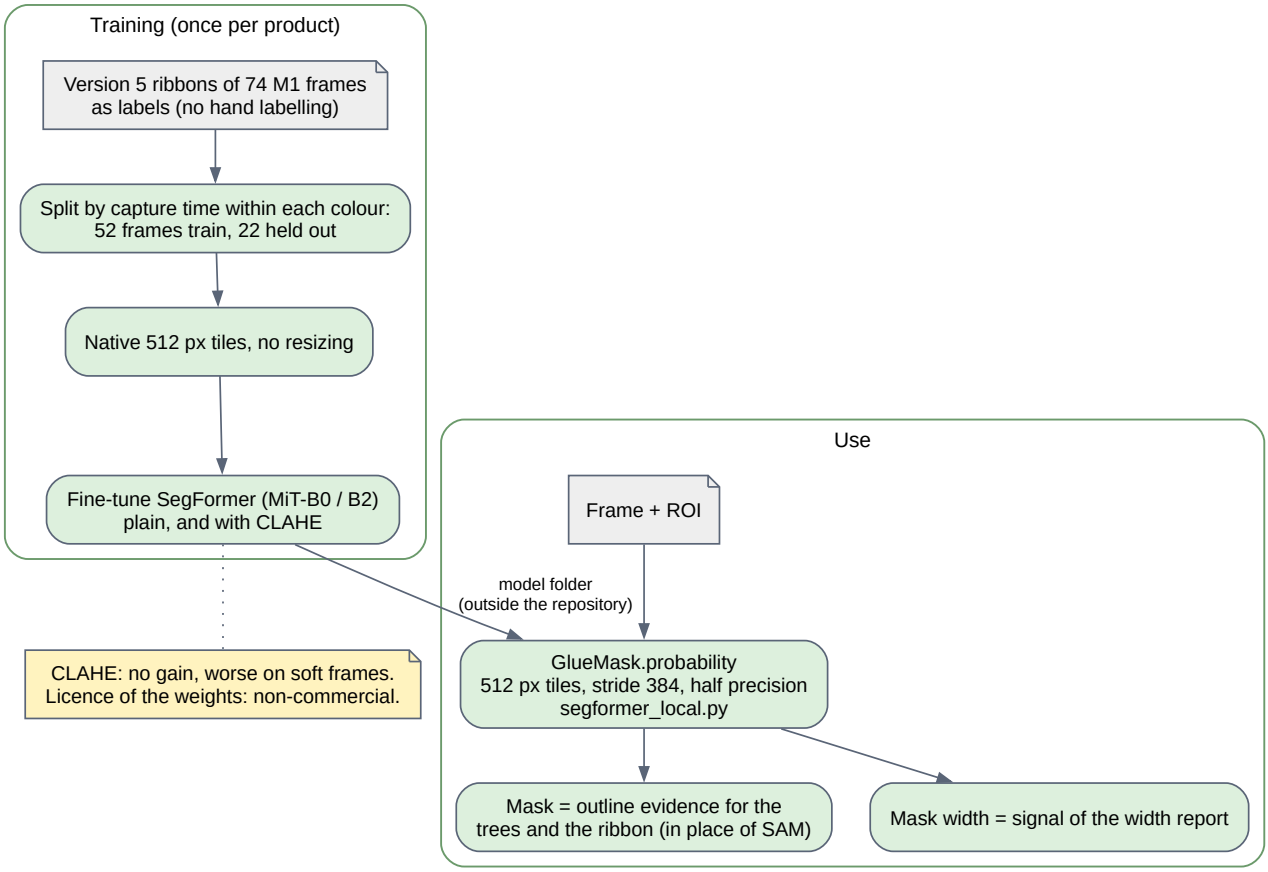


Figure 15.1 The segmenter is trained on the flow's own best outlines and then used as a second source of evidence.

SegFormer is a transformer segmentation network. It was fine-tuned to mark the glue band, using the version 5 ribbons of the M1 frames as labels, so nothing was labelled by hand.

	SegFormer-B0	SegFormer-B2
Parameters	3.7 M	27.3 M
Training	3000 iterations, batch 6, 7.5-16 min	4000 iterations, batch 3, 19 min
GPU memory, training / running	1.8 GB / 0.66 GB	3.3 GB / 1.9 GB
Time for a frame's mask	0.3-0.9 s	0.8-1.3 s

- 52 frames to train on, 22 held out, cut per colour by capture time so repeat shots stay on one side. 2 102 tiles of 512 px at native resolution; glue is 5.5 % of the pixels.
- Loss: cross-entropy plus Dice. Augmentation: flips, transposition, brightness jitter.
- The network outputs at a quarter of the input resolution.

15.2 The real test: frames of other pieces, never trained on

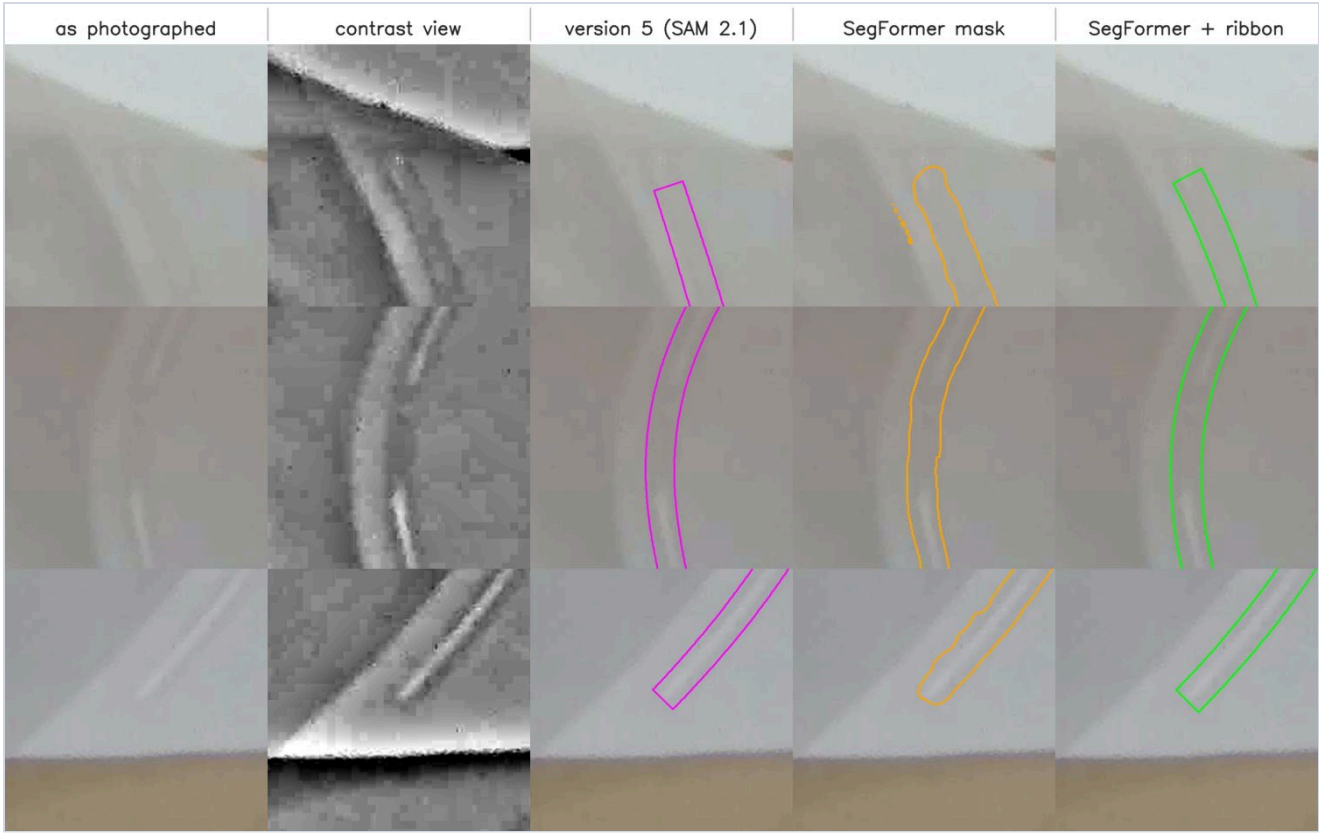


Figure 15.2 A soft white frame from another image set. Version 5 with SAM 2.1 leaves the bead at one end; the SegFormer mask follows it.

Soft white frames, 17 lines	Version 5 (SAM 2.1)	SegFormer-B0	B0 + CLAHE	SegFormer-B2
Width range, px	22-36	26.4-32.0	26.4-31.2	**28.0-32.0**
Outline source covers the line	0-84 %	60-95 %	0-87 %	**69-100 %**
Lines marked "check by eye"	9 of 17	1 of 17	1 of 17	1 of 17

It is the first method that handles the soft white frames, although it was trained on sharp frames only, four of them white.

15.3 CLAHE

Trained and run with CLAHE on lightness (clip 3, 16 x 16 grid): no gain on the M1 frames, every figure equal to within 0.03 px, and worse on the soft frames, where one line gets no mask at all. This agrees with every filter result for SAM and for the local detector: on these pictures a contrast filter does not help a segmenter, and on soft pictures it hurts.

15.4 Against SAM

	SAM 2.1 large (version 5)	SegFormer-B2
Needs a rough line first	yes	no
Needs training on this product	no	yes, about 20 minutes on this GPU
A new fabric colour or product	works untrained (three colours measured)	unknown; expect retraining
Soft frames	weak	good
Time for the outline	3-4.5 s	about 1 s
Licence of the weights	Apache 2.0	**non-commercial** (NVIDIA)

15.5 State

- Not in the Pipeline Lab. `segmentation/segformer_local.py` loads a trained model folder and is used by the width report (chapter 16).
- Models and training scripts are in `~/flod_experiments/segformer/`, outside the repository.
- Agreement with version 5 on held-out M1 frames shows it learned the task, not that it is more accurate: it was trained on those ribbons.
- Use in the product needs NVIDIA's agreement, or the same training on an openly licensed network such as the U-Net already used for the region model. That was not tried.

16 Width in millimetres and width changes

The user's request: taking the average strip as 4 mm, turn pixels into millimetres, find by a proper statistical method where a line is wider or narrower, show it on a page, and resolve changes of 0.1-0.2 mm. Code: `segmentation/width_stats.py`, `segmentation/segformer_local.py`, `tools/width_report.py`.

16.1 The method

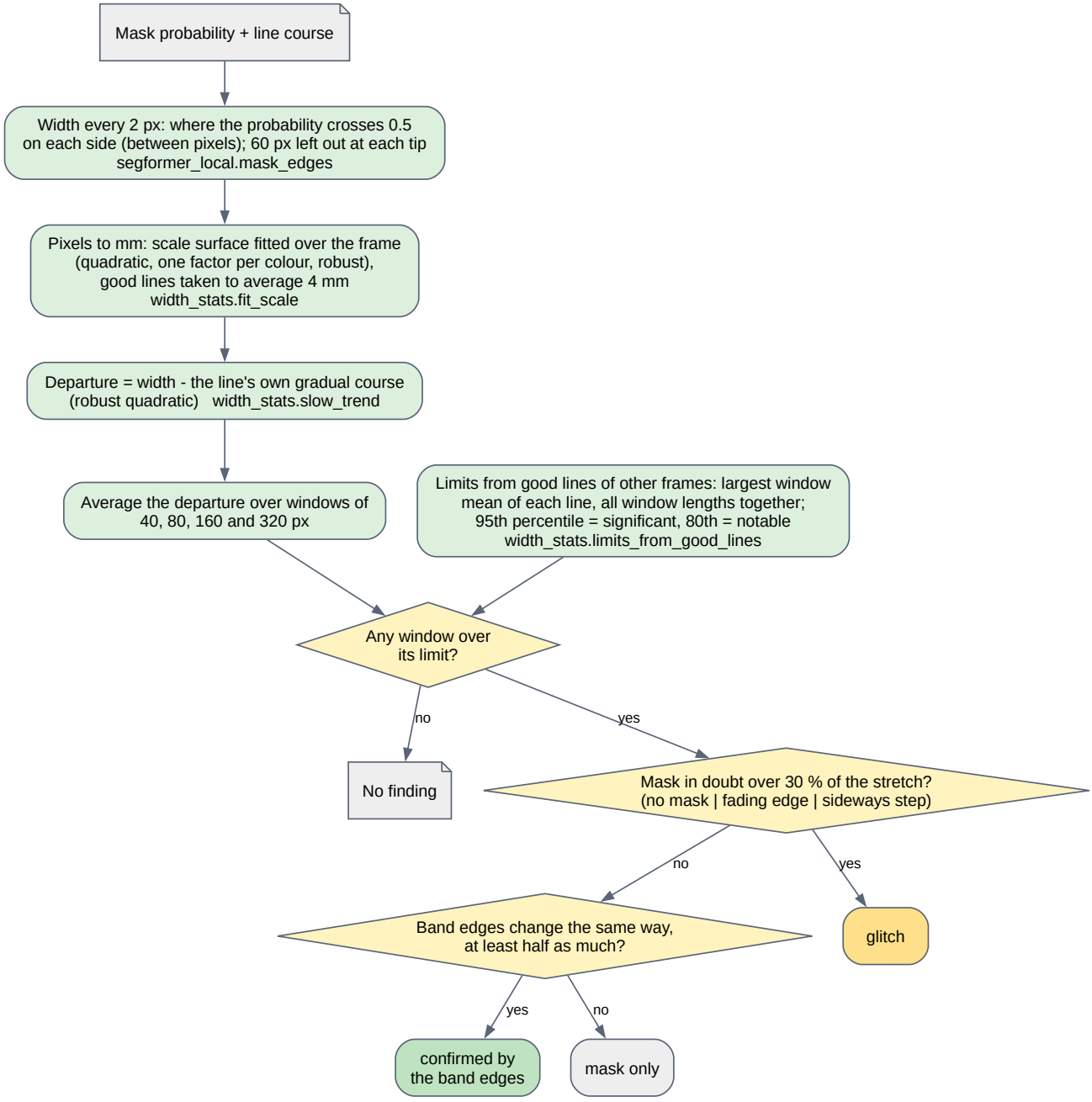


Figure 16.1 From a mask to a judged width change.

1. **Width** is read from the segmentation mask, pixel by pixel across the line: the two places where the probability of glue passes one half, found between pixels.

2. **Scale.** The camera is uncalibrated, so pixels per millimetre is *fitted*: a quadratic surface over the frame with a factor per fabric colour, fitted robustly to the logarithm of width, on the assumption that good lines average 4 mm. Mean 7.57 px per mm.
3. **Departure** is the width minus the line's own gradual course. A gradual change along a whole line is mostly the camera and is reported separately.
4. **Test.** The departure is averaged over windows of 40, 80, 160 and 320 px. A window is flagged when its mean exceeds a limit taken from good lines.
5. **Judge** each flagged stretch: mask glitch, confirmed by the band edges, or mask only.

16.2 Why the mask width, and why not three sigma

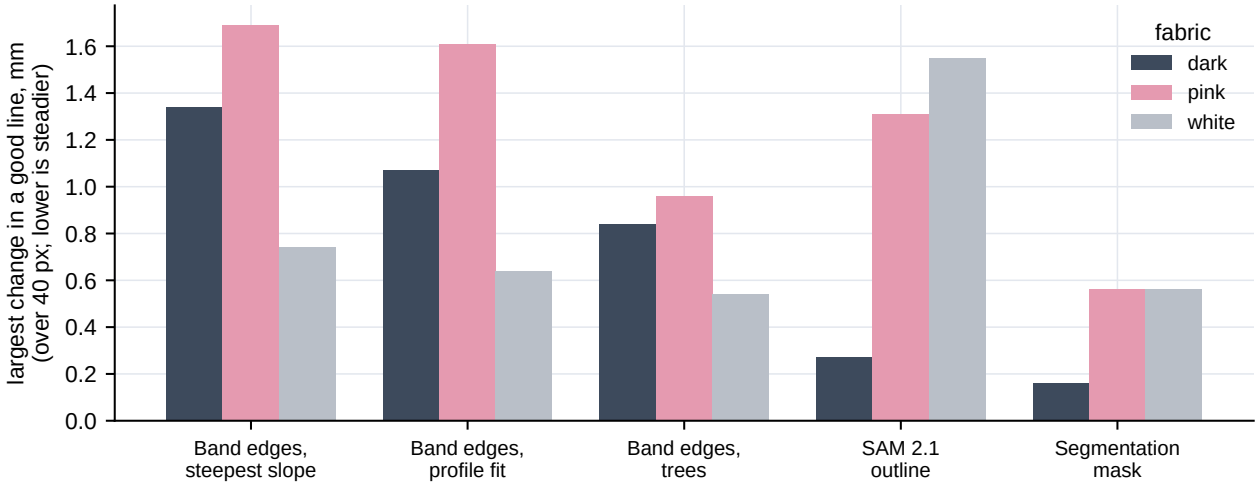


Figure 16.2 The largest change of width inside one good line, by where the width is read from. A steadier signal can find a smaller defect.

- All supplied lines are good, so the best signal is the one that varies least on them. Band-edge widths, the trees included, have a bump of about 0.8 mm somewhere on nearly every good line. The mask width is three to five times steadier.
- Limits from a normal table fail here. Neighbouring widths are correlated (0.8 at 10 px, 0.4 at 40 px) and the spread has heavy tails (2-9 % of readings beyond three robust standard deviations, against 0.27 % for a normal distribution). Three-sigma limits would flag most good lines.
- So the limits are **empirical**: for each line the largest window mean is found; the limit is a percentile of that over good lines, taken over all window lengths together. At the 95th percentile, 5 good lines in 100 are flagged by chance at any length.
- A line's limits come from other frames (five groups by capture time), and only from the 22 frames the mask model was not trained on.

16.3 What it reaches

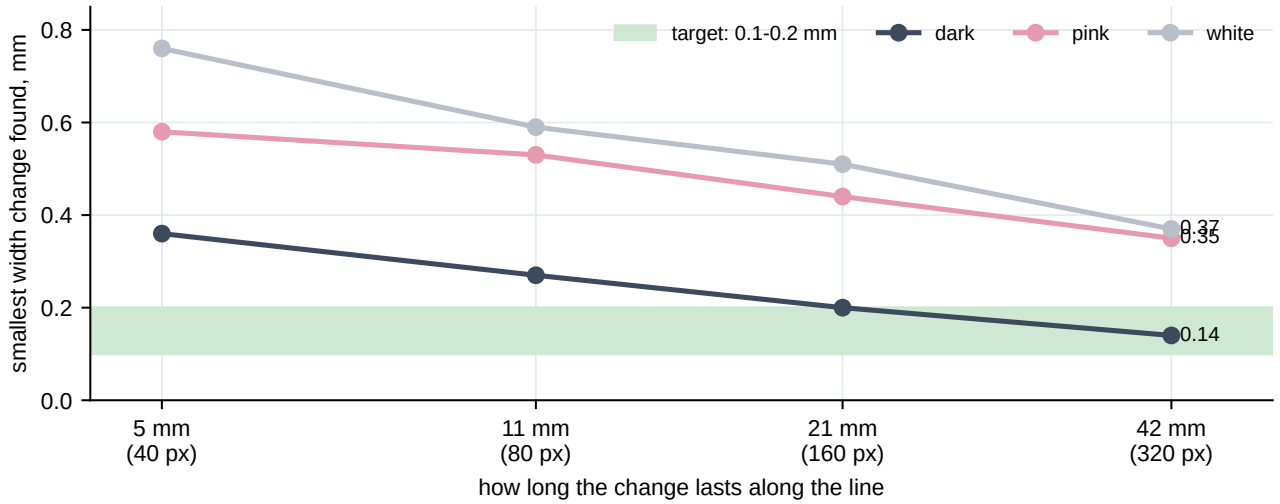


Figure 16.3 Smallest width change marked significant, by fabric colour and by how long the change lasts. The green band is the user's target.

	Dark	Pink	White
Line width after the scale, mm (mean +- SD between lines)	4.00 +- 0.06	4.02 +- 0.16	4.01 +- 0.08
Spread of 40 px readings along a good line, mm	0.048	0.168	0.130
Smallest change found, lasting 5 mm	0.36	0.58	0.76
... lasting 42 mm	**0.14**	0.35	0.37

- **The target is met on dark fabric for changes 20-40 mm long** (0.14-0.20 mm), and not met on pink or white, nor for short changes on any colour.
- The limit is set by the natural variation of good lines, not by the measurement: a single 40 px reading on dark fabric is steady to 0.05 mm.
- Check of the method: of 78 lines new to the mask model, 7 were marked significant (9 %; 5 % expected) and 10 more notable (13 %; 15 % expected).

16.4 Segmentation glitches

Check along a line	A glitch when	Found on 257 lines
No mask	the line is outside the mask for 3 readings or more	0 stretches
Mask fades out	the probability falls slowly at an edge, for 20 px or more	115 stretches, typically 4 mm
Mask steps sideways	the mask's middle leaves its own course beyond what good lines show	24 stretches, typically 19 mm

Width changes found	Glitch	Also in the band edges	Mask only
Significant (39)	0	33	6
Notable (60)	2	36	22

Most flagged changes are real variation of good glue, seen by a second reading that owes nothing to the mask.



Figure 16.4 A frame of the width report. Each line is tinted by its departure (blue narrower, red wider); yellow marks a glitch stretch; the label is the line's mean width in mm.

16.5 What it does not cover

Case	Found?	Why
A change lasting 40 px (5 mm) or more	yes	every window length has its own limit
A bump shorter than 40 px	only if large	the shortest window averages it: a 20 px bump counts at half its size
A step in width	late, and not located	no test compares neighbouring readings
The mask changes width without fading or moving sideways	as "mask only", not as a glitch	no glitch check looks at width
A real sudden defect that the mask rounds off	no	the model was trained on even ribbons and outputs at a quarter resolution

> **Unproven where it matters.** The limits control how often a *good* line is flagged. How often a *defective* line is caught has not been measured, because no defective sample exists; and a mask trained on even-width labels may smooth a real defect away.

17 Research tools

17.1 The map

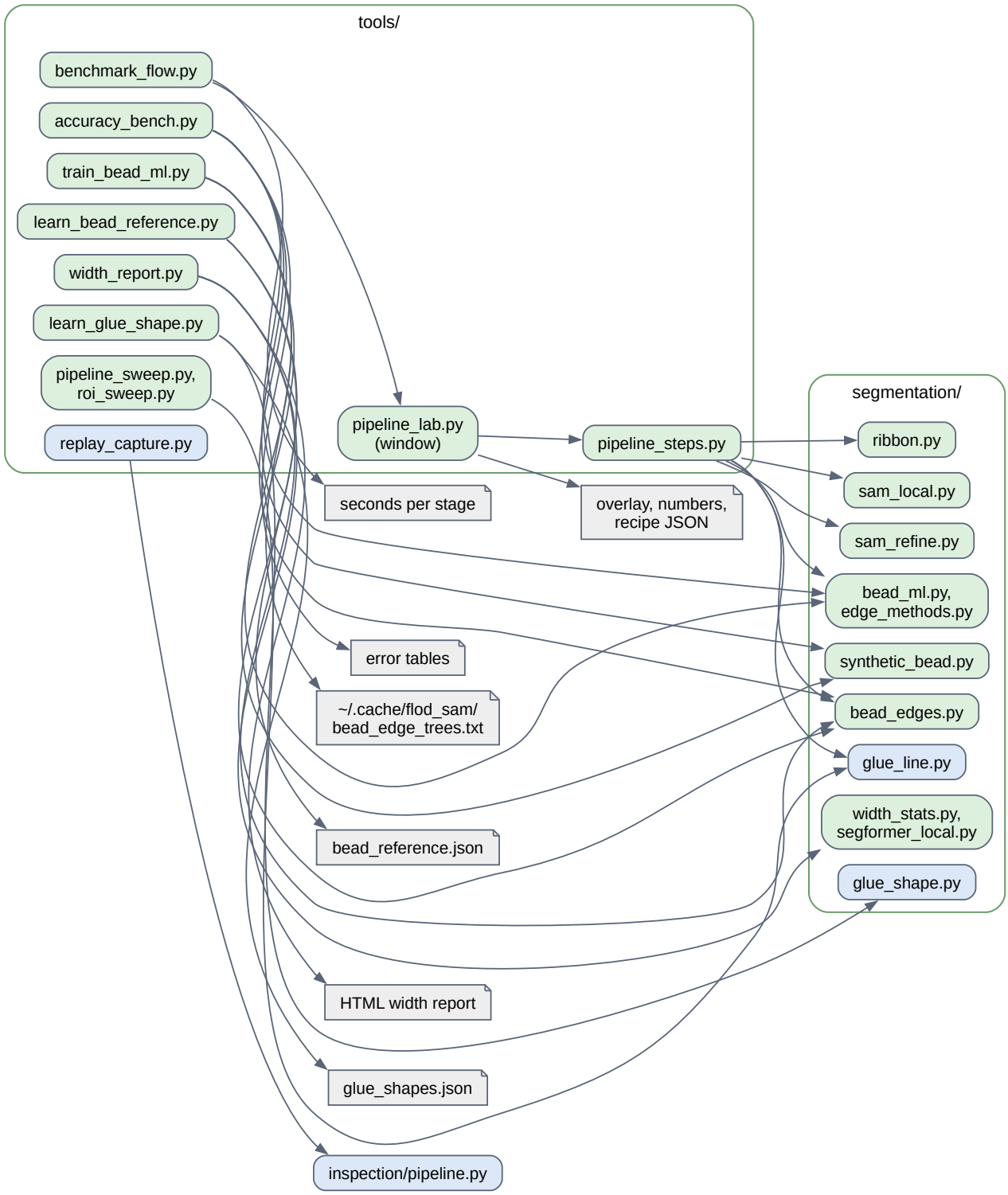


Figure 17.1 Tools, the modules they drive and what they write. None of the green boxes are used by the inspection application.

17.2 The Pipeline Lab

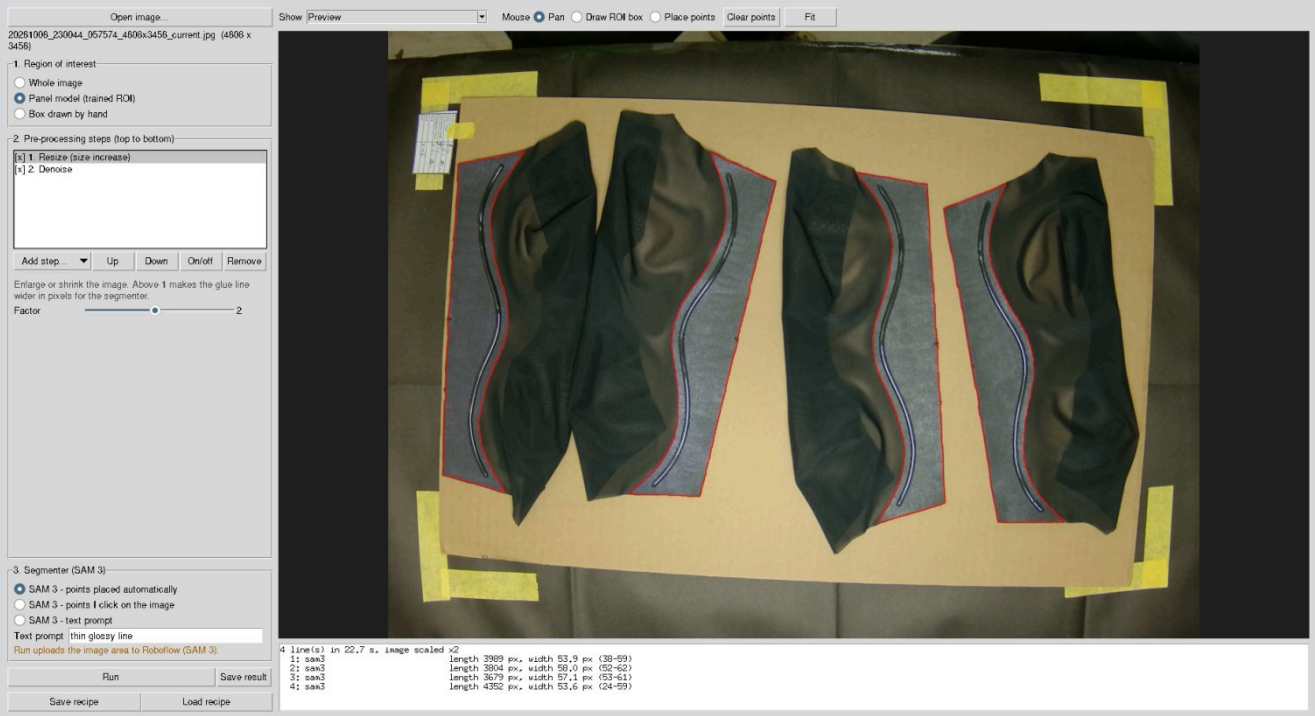


Figure 17.2 The Pipeline Lab window in an early state: region of interest, pre-processing steps and segmenter on the left; the picture and the per-line numbers on the right. Later options (where SAM runs, asking once, the ribbon, how edges are placed) were added below these.

`tools/pipeline_lab.py` is a window for trying things on any image. It is separate from the application and changes nothing in it. A set-up can be saved and loaded as a recipe (JSON), and `run_job(image, recipe, ...)` runs the same flow from a script.

Recipe key	Values	Meaning
`roi`	`whole`, `model`, `box`	region of interest
`pipeline`	list of steps	pre-processing, top to bottom (none is recommended)
`segmenter`	`local`, `sam3`	local detector only, or SAM with prompts
`point_source`	`lines`, `clicks`	SAM's points placed along the rough lines, or clicked by hand
`sam_runs`	cloud, `local`	Roboflow, or this PC's GPU
`local_model`	`sam2.1-large`, `-base-plus`, `-small`, `-tiny`, `sam3`, `mobile-sam`, `fastsam-s`, `fastsam-x`	which local model
`ask_once`	true / false	skip the repeat askings
`ribbon`	true / false	fit the smooth ribbon
`edge_method`	`slope`, `profile_fit`, `trees`	how the band's edges are placed

17.3 Commands

All from the repository root, with `~/flod_experiments/gpu/bin/python` on the development PC.

Purpose	Command
Open the window	`python tools/pipeline_lab.py [image]`
Time each stage	`python tools/benchmark_flow.py --sam local`
Score edge methods against painted truth	`python tools/accuracy_bench.py --runs DIR`
Train the edge-correcting trees	`python tools/train_bead_ml.py --runs DIR`
Learn the black-fabric reference	`python tools/learn_bead_reference.py NAME frames...`
Learn a wave template for the shape condition	`python tools/learn_glue_shape.py NAME frames/*.png`
Write the width report	`python tools/width_report.py --runs DIR --out DIR --mask-model DIR --held-out FILE`
Replay saved images through the application's pipeline	`python tools/replay_capture.py images/*.jpeg`
Run the tests	`python -m unittest discover -s tests -p "test_*.py"`

`DIR` is a folder of cached runs (one file per frame, as in `~/flod_experiments/segformer/runs`). Each script prints its own options with `--help`.

17.4 What lives outside the repository

Place	Content
`~/flod_experiments/gpu/`	Python environment: PyTorch with CUDA, transformers, onnxruntime, ultralytics, scikit-learn, lightgbm
`~/flod_experiments/segformer/`	cached version 5 runs of the 74 frames, training scripts, three trained models
`~/flod_experiments/main_model/`	read-only copy of `inspection/local/` from `main`, and the scripts that scored it
`~/flod_experiments/out/`	result pages, each with an `index.html`
`~/.cache/flod_sam/`	the trees model, MobileSAM and FastSAM weights
`enhancement_images/` (in the folder, git-ignored)	the sample frames

Model weights (`*.onnx`, `*.pt`) and the image sets are git-ignored. The region model file must be supplied separately.

17.5 Tests

`tests/` holds 366 tests on this branch; all pass. Those for the research code are `test_pipeline_lab.py`, `test_accuracy_bench.py` and `test_width_stats.py`. They use synthetic images, never a camera, the cloud or a GPU.

PART D

The application on main (read-only)

18 Architecture of the application on `main`

> **How this part was written.** `main` was read at commit `485c4b6` ("updater_added", 2026-10-09) with `git show`; it was not checked out, merged or changed. The description is **read from the source**. Only the glue model in `inspection/local/` was run, at commit `4a0680f`; its code did not change between the two commits. This part goes out of date when `main` moves. The author of this code is Obhash Mannapperuma.

18.1 What changed against the common base

Sixteen commits; 56 files; about 5 100 lines added and 520 removed.

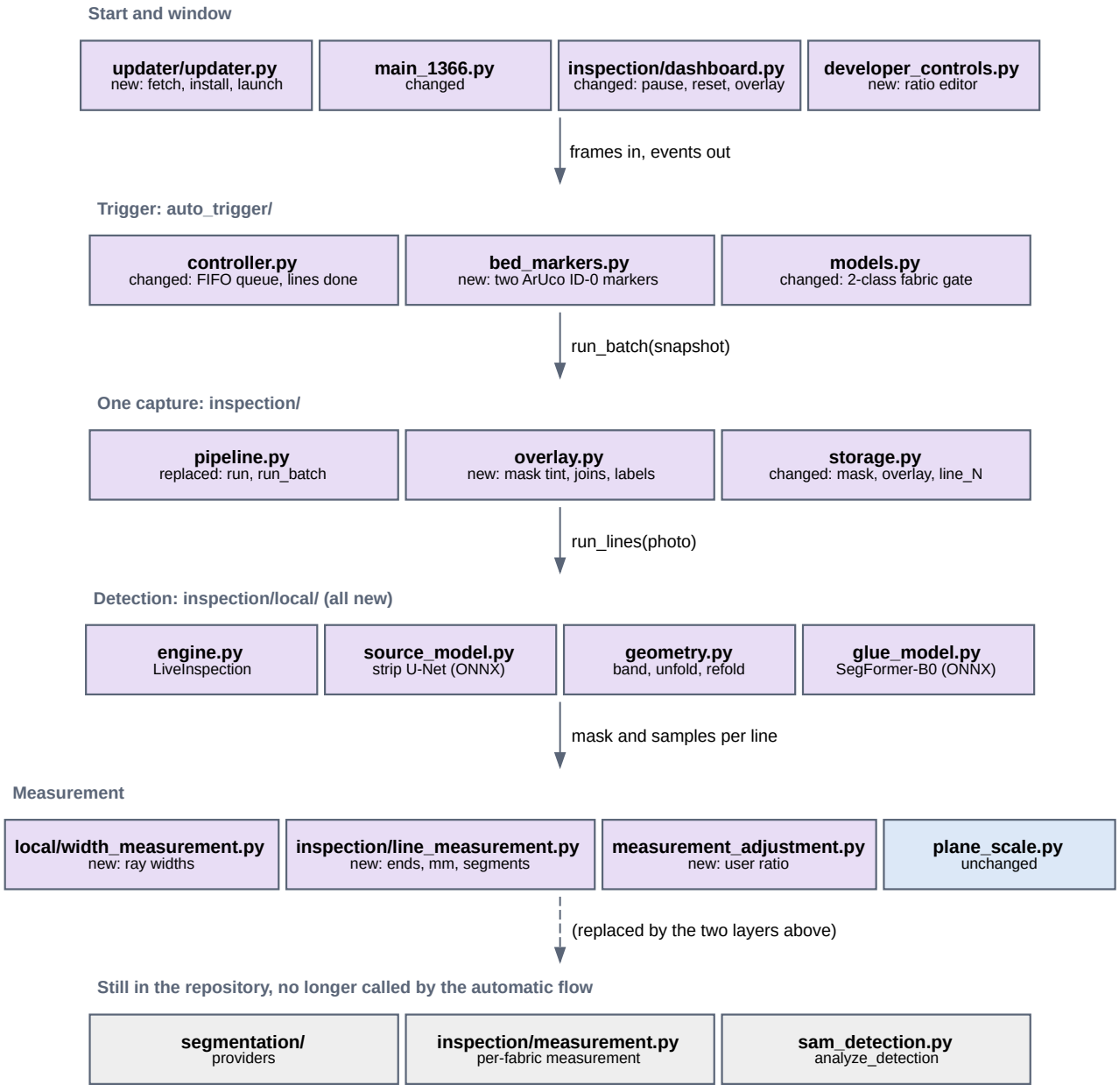


Figure 18.1 The application on main by layer. Purple: added, changed or replaced there. Grey: still in the repository but no longer called by the automatic flow.

Area	Change
Detection	The cloud workflow provider is no longer used. `inspection/local/` finds the glue with two ONNX models on the CPU, offline.
Pipeline	`inspection/pipeline.py` is rewritten: `run` (rightmost line) and `run_batch` (every line a capture still owes). It takes an *inspector*, not a segmenter.
Measurement	`inspection/local/width_measurement.py` and `inspection/line_measurement.py` replace `strip_analysis` for the automatic flow.
Trigger	The hand cycle stays. Added: a FIFO queue of captures, a record of lines already inspected, and a reset when two bed markers become visible.
Fabric gate	A two-class classifier (`full_fabric` , `no_fabric`); accepted when the first score is larger.
Screen	Startup loading screen, pause, live overlay of hands and markers, per-line results, mask tint and segment joins on the overlay.
Developer controls	A user distance ratio that multiplies millimetre results, kept apart from the calibration.
Camera	Rotation, settle time before locking controls, changed default index.
Updater	`updater/updater.py` : a starter program that fetches `main` , installs requirements and launches the app.
Models in the repository	`segformer_b0.onnx` (15 MB), `strip_unet_resnet34.onnx` (98 MB), `keras_model_2c.h5` . On this branch model weights are git-ignored.

18.2 Settings added or changed

Setting	Value	Meaning
`local_inspection.source_model`	`inspection/local/models/strip_unet_resnet34.onnx`	the panel-strip model
`local_inspection.glue_model`	`inspection/local/models/segformer_b0.onnx`	the glue model
`local_inspection.source_width`	1152	input width of the strip model
`local_inspection.offset_pixels`	100	width of the band inside the curved side
`inspection.end_exclusion_percent`	5.0	share of each end of a line left out of grading
`auto_trigger.marker_confirm_seconds`	0.5	how long both bed markers must be seen
`preview.show_overlay` , `marker_detect_width`	true, 1280	live overlay of hands and markers
`capture.auto_lock_settle_seconds`	5.0	wait before camera controls are locked
`capture_storage.max_sets`	200 (was 1000)	capture folders kept
`auto_trigger.accepted_labels` , `fabric_min_confidence`	removed	replaced by the two-class gate
`cameras[0].rotation`	180 (was 0)	the camera is mounted the other way up

18.3 The updater

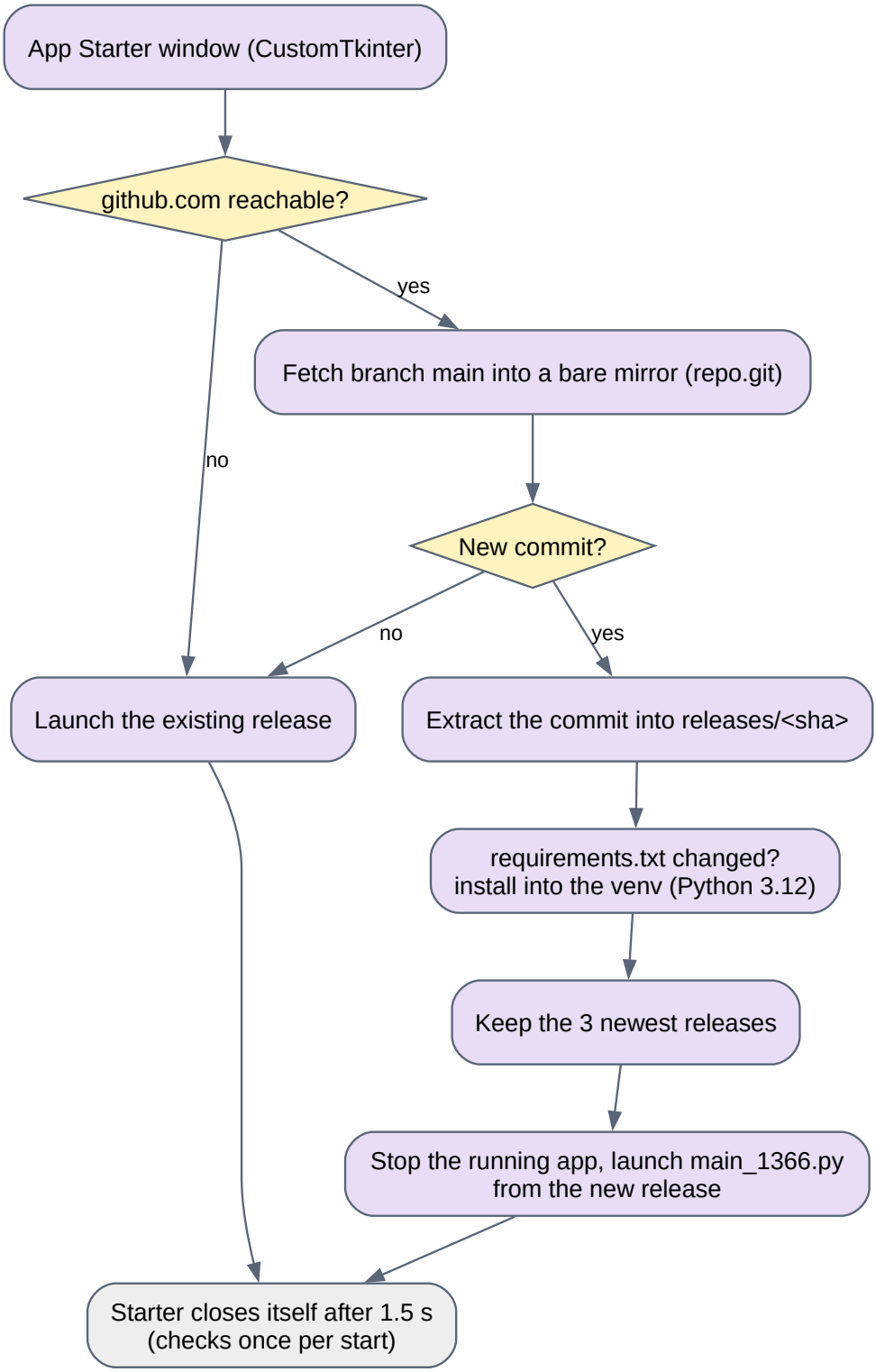


Figure 18.2 The updater's loop, read from updater/updater.py. It was not run.

It keeps a bare mirror of the repository. **Once, when it is started,** it fetches `main`; if the commit is new it extracts it into its own release folder, reinstalls requirements when that file changed, keeps the three newest releases, launches `main_1366.py` and closes itself 1.5 s later. When the PC is offline it starts the release it already has. The 60-second interval in the file is not used.

> **Consequence for branches.** The updater installs whatever is on `main`. Anything merged into `main` reaches a PC the next time the starter is run there. The research on `tooling/pipeline-lab` is not on `main` and is not distributed by it.

Findings of the review (`docs/glue-line/MAIN_BRANCH_REVIEW.md`), all from reading the code:

#	Severity	Finding
U1	High	A failed package install is recorded as done: the new build starts on the old packages and the install is not retried until another commit arrives
U2	High	`config.json` lives in the build folder, so camera index, rotation, tolerance and offset return to the committed values at every update. Calibration and captures are in Documents and are kept
U3	High	Every push to `main` is delivered with no test run, tag or way back; 2 of the 365 tests on `main` fail at `485c4b6`
U4	Medium	`git` and `pip` have no time limit: a stalled network or a credential prompt hangs the starter
U5	Medium	An environment that is missing or rebuilt is left without packages unless `requirements.txt` changed
U6	Medium	Errors in the fallback launch are not caught; the window stays on its last message

19 The inspection pipeline on `main`

19.1 From a photo to graded segments

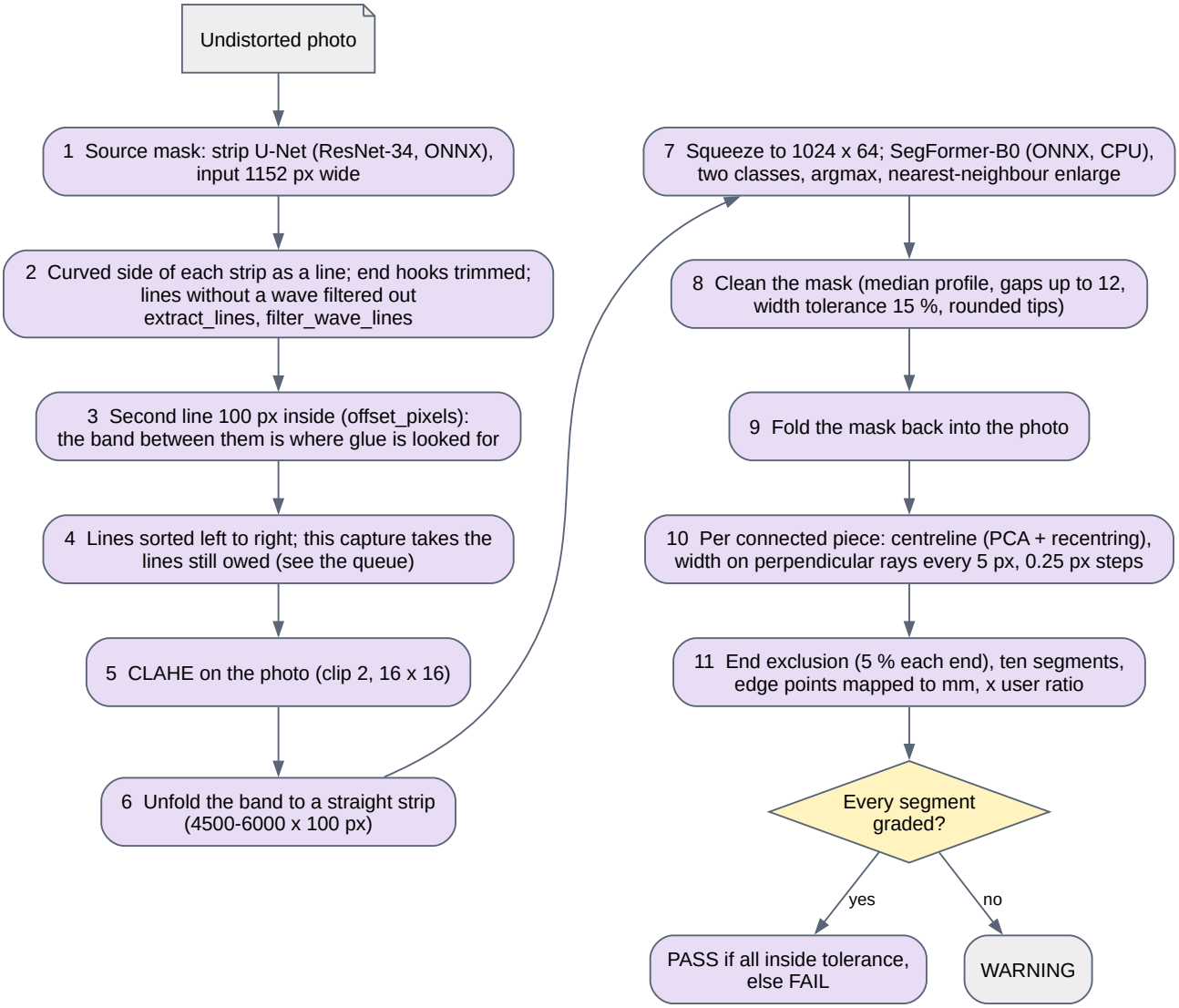


Figure 19.1 The pipeline of LiveInspection and measurement_record, read from the source.

Stage	Code on `main`	Detail
Source mask	`local/source_model.py`	the same strip U-Net as the research's region model, as ONNX on the CPU
Lines	`local/geometry.py`: `extract_lines`, `filter_wave_lines`	the curved side of each strip; hooks at the ends trimmed; lines without a wave refused
Band	`offset_lines`, `region_mask`	a second line `offset_pixels` (100) inside; glue is looked for between the two
Enhance	`enhance_image`	CLAHE, clip 2, 16 x 16 tiles
Unfold	`unfold_maps`	the band straightened along its middle line: 4500-6000 x 100 px for a line 2200-2900 px long, because the inner edge of the band zigzags (review finding M3)
Glue model	`local/glue_model.py`	band squeezed to 1024 x 64; SegFormer-B0, two classes; class chosen per pixel, enlarged by nearest neighbour
Clean	`clean_mask`	profile smoothing, gaps up to 12, width tolerance 15 %, rounded tips
Refold	`mapped_mask`	the mask mapped back into the photo
Measure	`local/width_measurement.py`	per connected piece: centreline, then width on perpendicular rays every 5 px in 0.25 px steps, with a sub-pixel crossing
Grade	`inspection/line_measurement.py`	5 % of each end left out; ten equal segments; both edge points of each sample mapped to mm; times the user ratio

Pieces of glue are measured separately; a gap is never bridged. The status is `PASS` when every segment's average is inside target +- tolerance, `FAIL` when a graded segment is outside, and `WARNING` when any segment could not be graded.

19.2 How long it takes

Measured on the development PC (i5-13420H, no GPU used), nine frames, 32 lines, with the model code of `main` run unchanged.

	Seconds
One capture measuring one line, as the app runs it	1.75 (1.6-1.9)
One frame with all its lines (2-4)	3.4 (2.4-4.1)
Loading the two models, once	0.65

Stage	Per frame, s	Per line, s	Note
Strip U-Net (source mask)	0.91		once per frame
Source lines: extract, wave filter, offset	0.32		once per frame
CLAHE on the whole photo	1.08	0.31	computed again for every line
Unfold, crop, refold	0.10	0.03	
SegFormer-B0	0.09	0.026	
Clean mask	0.31	0.09	
Width on rays	0.31	0.09	
Not timed singly	0.30		remaps, connected pieces, overlay

> **Finding:** the glue model is 26 ms per line. The largest cost is CLAHE, because the whole 16-megapixel photo is enhanced again for each line. Once per frame would save about 0.3 s for every line after the first.

The trigger wait (hands absent for 1.0 s), the fabric check, the undistortion and the saving were read, not timed. The Windows inspection PC was not timed.

19.3 Review findings

The full review, with file and line for each point, is `docs/glue-line/MAIN_BRANCH_REVIEW.md`.

#	Severity	Finding	Evidence
M1	High	Only the average of each 25 mm segment is graded, at +- 1.0 mm. A 2 mm blob must be 4.2 mm wider than the strip to fail; 5 mm long, 2.9 mm wider	12 real masks, widened by drawing
M2	High	A line in two in-tolerance pieces passes; a line with no glue found shows WARNING, not FAIL	code; soft frames
M3	Medium	The band's inner edge zigzags: 18-20 % of its steps go backwards, so the strip is 4500-6000 px long	8 lines
M4	Medium	Moving the photo one pixel changes a segment average by 0.35 px on average and 4.0 px (0.53 mm) at worst	37 frames
M5	Medium	One model row is 0.2 mm across the line; the class is chosen before enlarging	model output 64 x 1024
M6	Medium	Soft white frames: good lines fail on one segment, lines break, extra lines appear	3 frames
M7	Medium	Changing the camera rotation does not invalidate the calibration	read

Taking the band's normal from smoothed edge points (model not retrained) leaves segment evenness as it is (worst segment 5.35 against 5.21 px) but improves repeatability: the worst change under a one-pixel shift falls from 4.0 to 1.1 px.

19.4 Trigger, queue and reset

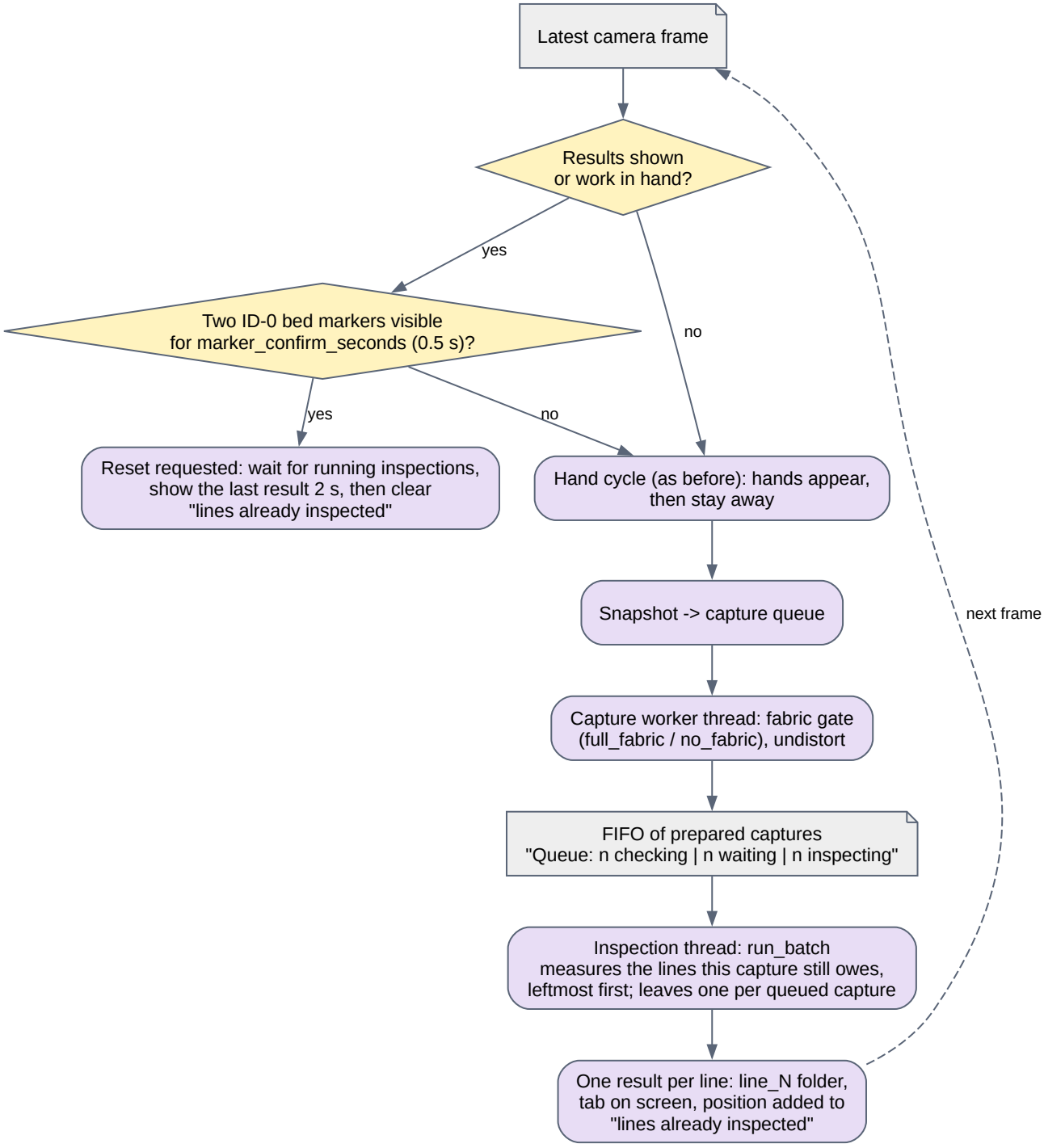


Figure 19.2 How a capture is taken and how the bed is reset, read from auto_trigger/controller.py.

- **Lines are counted from the left** and a position names the same physical line in every capture of a cycle, because panels are only ever added on the right.
- Each capture measures the lines not yet inspected, leftmost first, and leaves one for each capture already queued behind it. A line that failed is offered again.
- **Reset.** When results are showing or work is in hand, two ArUco markers with ID 0 on the bed, both visible for half a second, mean the bed has been cleared. The controller waits for running inspections, shows the last result for two seconds and forgets which lines were inspected.

- The fabric check and the undistortion run on a separate worker thread so the frame loop is not held up.

19.5 What is saved

One folder per capture, as on this branch, with a `line_N` folder inside for each line measured: its mask, its overlay and its measurement. The record's method is `local_onnx_lines`.

19.6 Compared with the application on this branch

	This branch (Part B)	`main`
Finds glue with	a provider: cloud workflow by default	two local ONNX models
Needs a connection	yes, by default	no
Unit of result	one fabric	one line
Several panels	all in one capture, up to four	one more line per capture, tracked across captures
Cleared by	fabric classifier saying "no fabric"	bed markers becoming visible
Ends of the line	half a width trimmed	5 % of the length left out
Statuses	PASS, FAIL and four ungraded kinds	PASS, FAIL, WARNING
Width read by	rays across the provider's polygon	rays across the cleaned mask

20 The two detection pipelines compared

20.1 Side by side

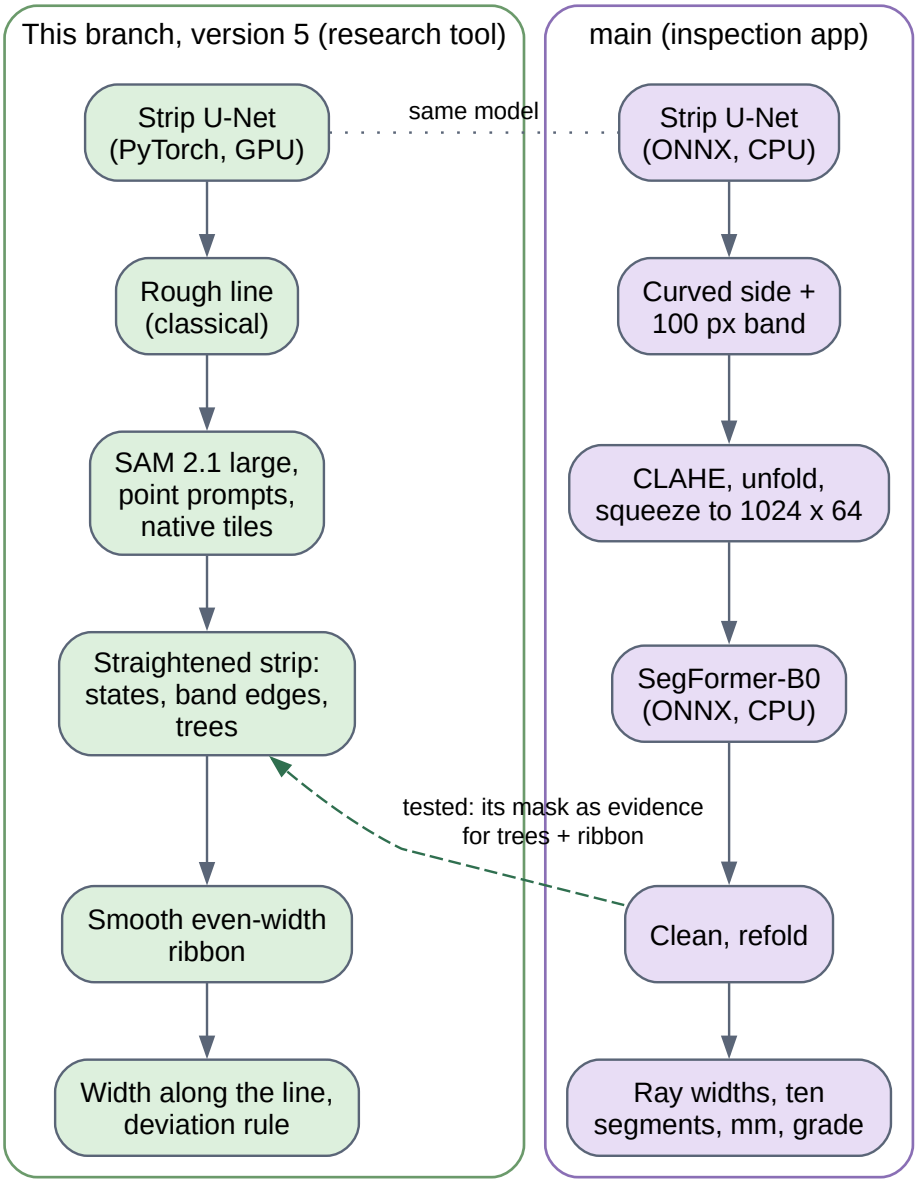


Figure 20.1 The research flow (version 5) and the pipeline on main. Both begin with the same strip model and both straighten the band; they differ in what reads it.

	Version 5 (research tool)	Glue model on `main`
Finds the object with	SAM 2.1, point prompts, native tiles	SegFormer-B0 trained for this glue
Resolution at the glue	native: band about 30 px wide	unfolded band squeezed 4.4-5.8 times along the line and 1.6 times across (one model row is 0.2 mm)
Contrast filter	none (measured: filters do not help)	CLAHE
Edges	band edges corrected by trained trees	boundary of the cleaned mask
Shape knowledge	smooth even-width ribbon	cleaning with a width tolerance
Hardware	GPU	CPU
Time per frame, same nine frames	4.3 s	3.4 s
Time for one line	1.2 s (every line of the frame is done together)	1.75 s for a capture measuring one line
Needs training per product	no	yes

20.2 Time on the same frames

Nine M1 frames (4 dark, 5 pink), 32 lines; both found the same lines on every frame.

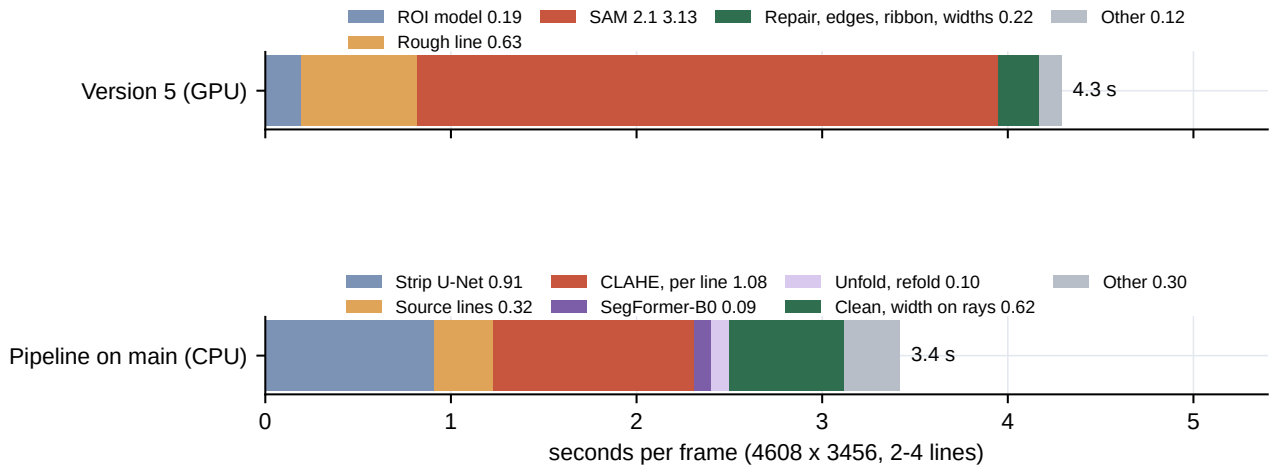


Figure 20.2 Seconds per frame by stage, mean of the same nine frames. Version 5 on the GPU, the pipeline on main on the CPU.

	Version 5 (GPU)	Pipeline on `main` (CPU)
One frame, all lines: mean (range)	4.3 s (2.4-5.4)	3.4 s (2.4-4.1)
Largest stage	SAM, 3.1 s (73 %)	CLAHE, 1.1 s (32 %), then the strip U-Net, 0.9 s
The stage that reads the glue	SAM 2.1: 3.1 s per frame	SegFormer-B0: 0.09 s per frame
Everything after the glue is found	0.2 s (repair, states, trees, ribbon, widths)	0.6 s (clean, width on rays)
Loading the models	16 s	0.65 s
GPU memory	983 MB	none

- The two take about the same time, on unequal hardware: `main` needs no GPU.
- `main` reads the glue 30 times faster than SAM does. Its time is spent around the model, not in it.
- What the research adds after the outline costs 0.2 s per frame. Put behind the model on `main` (the tested combination below) the estimate is 3.6 s per frame without a GPU; that combination was not timed as one program.

20.3 Measured on the same 74 frames

The model was extracted to `~/flod_experiments/main_model/` and run unchanged on all 74 M1 frames (257 lines).

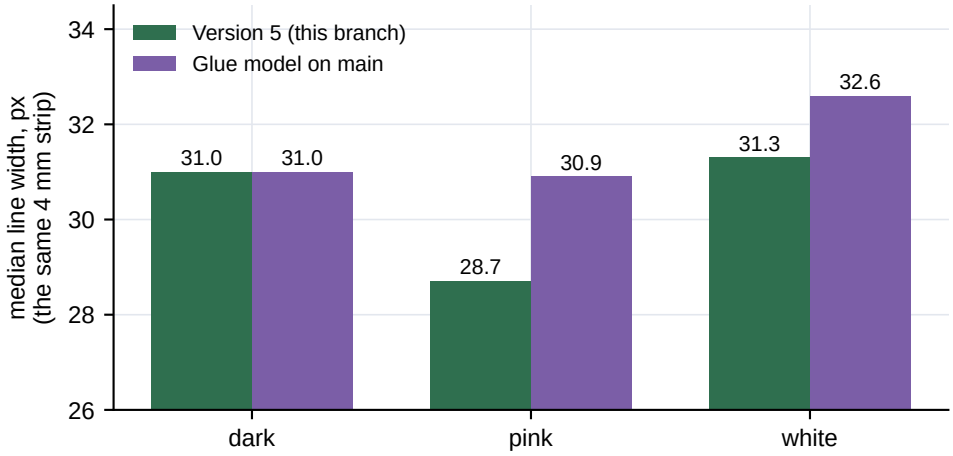


Figure 20.3 Median line width by colour. The same 4 mm strip should read the same on every colour.

	Dark	Pink	White
Lines found, as one piece	126 of 126	113 of 113	18 of 18
Width, median px: main model	31.0	30.9	32.6
Width, median px: version 5	31.0	28.7	31.3
Centre line from version 5's, px	0.68	0.81	0.80
Width noise from sample to sample, px RMS	1.25	1.29	1.35
Largest segment average off the line's mean, px: mean / worst	1.3 / 4.7	3.2 / 5.5	1.6 / 3.1
Line width between repeat shots, SD px (version 5)	0.12 (0.07)	0.13 (0.21)	-

- **It works and can be used.** Every line found as one piece, no errors, 3.4 s per frame without a GPU.
- **Its widths are equal on the three colours**, which is what one nozzle should give. Version 5 reads pink 2.6 px (about 0.3 mm) narrower. Which is right on pink is not known.
- **Its weakness is evenness along a line.** On good pink lines one of the ten segment averages is up to 5.5 px (0.7 mm) off the line's mean. A tolerance tighter than that would reject good lines.
- On soft white frames of other sets it finds the lines, but with readings far off and lines in two or three pieces.

20.4 The one combination that was tested

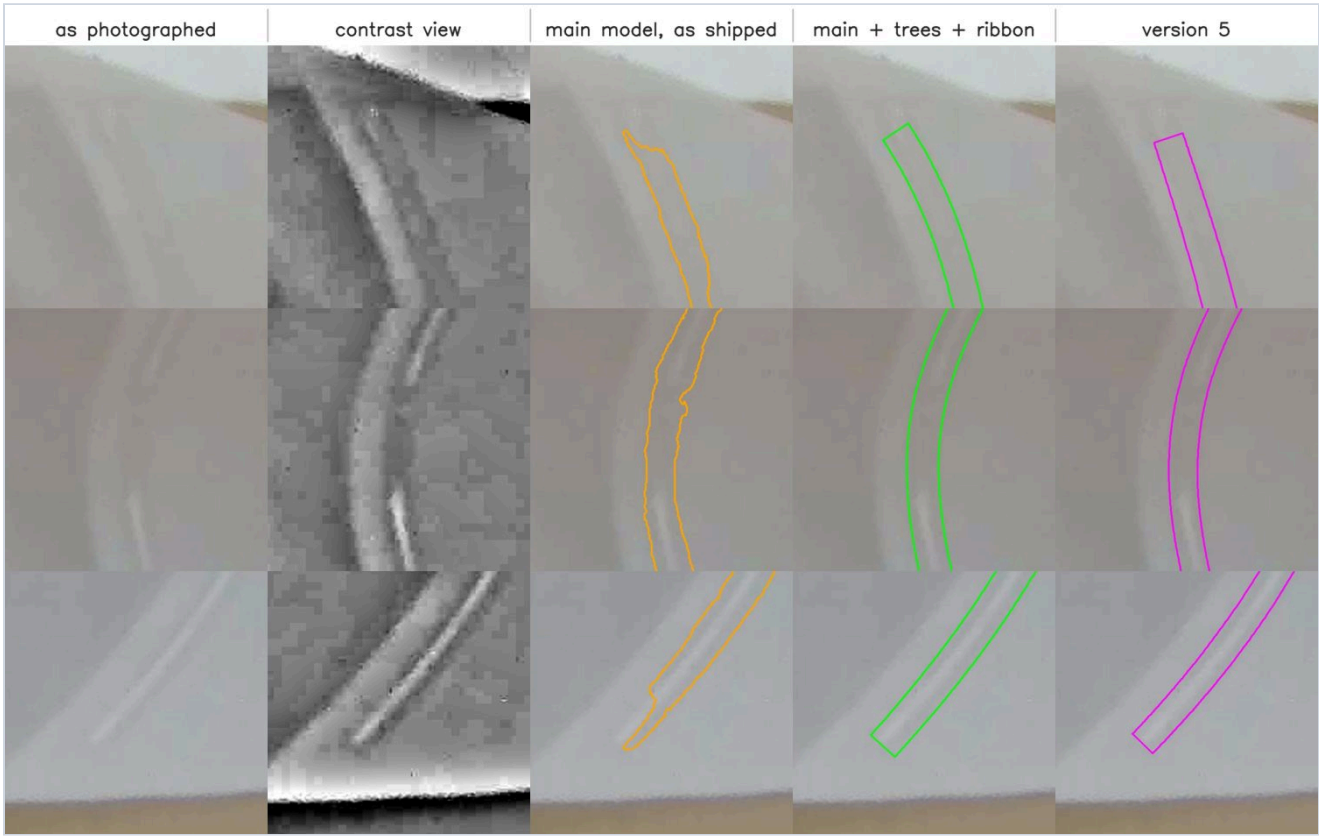


Figure 20.4 A soft white line. The main model's mask as shipped; the same mask finished with the trees and the ribbon; version 5.

Update to the main model	Result	Verdict
Its mask as evidence for the trees and the ribbon (no retraining)	centre 0.15-0.43 px from version 5; a 17.5 px reading becomes 26.0 px; two false lines refused	**recommended**
Enlarge the scores smoothly before choosing the class	edges 1.01 to 0.85 px on dark	harmless, small
Retrain in the same format on version 5 ribbons	not more even; worse on soft frames	not recommended
Retrain without CLAHE at 2048 x 128	soft frames much steadier; noisier on M1	promising; needs more data

20.5 What merging would involve

Not done. Eleven files were changed on both branches since the common base and would conflict or need care: `inspection/pipeline.py`, `inspection/result_view.py`, `camera_handler.py`, `config.json`, `config_validation.py`, `requirements.txt`, `strip_analysis.py`, `sam_detection.py`, `tests/test_automatic_inspection.py`, `tests/test_camera_runtime.py`, and `AUTOMATIC_INSPECTION.md` (rewritten on `main`, moved into `docs/` here; `CONFIGURATION.md` was also edited there and moved here). The research modules themselves (`segmentation/bead_edges.py`, `ribbon.py`, `bead_ml.py`, `width_stats.py` and the tools) touch nothing on `main` and would come across unchanged.

PART E

State of the work

21 Decision guide

21.1 Which method for which need

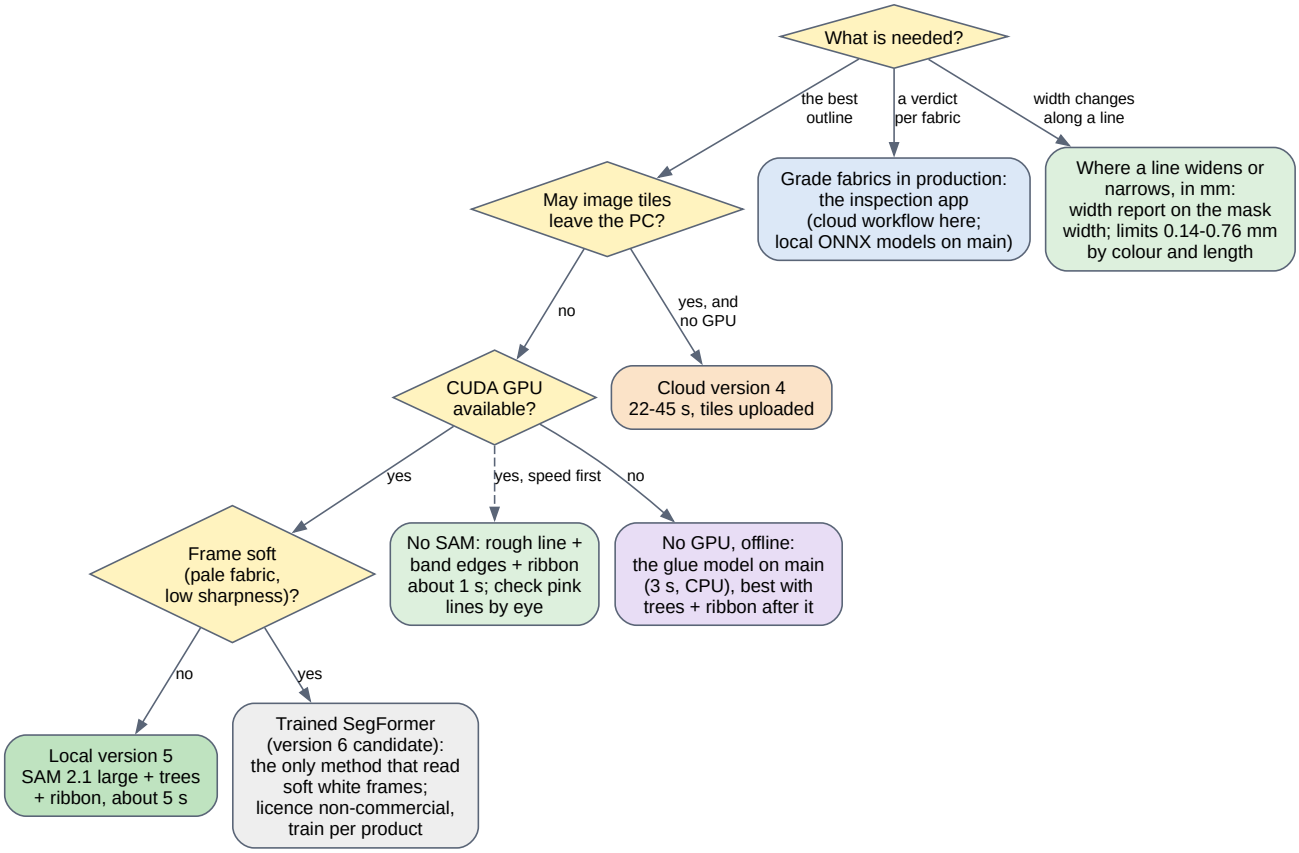


Figure 21.1 A summary of the findings as a decision tree. It is a guide for a person; no code selects a method this way.

21.2 The options in one table

Option	Where	Time per frame	Uploads	Strong at	Weak at	Judged by the user
Inspection app, cloud workflow	this branch	set by the connection	yes	whole cycle, any fabric	needs a connection; grades ten coarse segments	in use before this work
Inspection app, local ONNX	`main`	3 s, CPU	no	offline, no GPU, equal widths on all colours	evenness along a line; soft frames	by its author
Version 4	Pipeline Lab	22-45 s	yes	the reference result	slow	**best so far**
Version 5	Pipeline Lab	about 5 s, GPU	no	same result as 4, faster, more accurate edges	pink width in dispute; soft white frames	not yet
No SAM	Pipeline Lab	about 1 s	no	speed	19 pink lines need a look	not yet
Version 6 candidate (SegFormer-B2)	scripts only	about 1 s for the mask	no	soft white frames	licence; training per product	not yet
Main model + trees + ribbon	scripts only	3 s + under 1 s	no	even width from a CPU model	not built into either app	not yet
Width report	script	on cached runs	no	where a line widens or narrows, in mm	short and sudden changes; pale fabric	not yet

21.3 Rules that held throughout

- Work at full resolution. A reduced frame leaves the bead 2-5 px wide.
- Confine everything to the panel region, one line at a time.
- Do not filter the picture before a segmenter.
- Average along the line, never across it.
- Use two independent readings and report a width change only where they agree.
- Fit the shape the glue is known to have; report what departs from it.
- Measure error against something the method cannot satisfy by construction.
- New behaviour is opt-in, and the default reproduces the old result exactly.

22 What is verified, what is not, and next steps

22.1 Verified by measurement on the sample frames

Statement	Evidence
Every glue line of the 74 M1 frames is outlined	257 of 257 lines, versions 4 and 5, and the model on `main`
The local GPU flow gives the cloud flow's result	centre lines 0.1-0.5 px apart; 257 of 257 "ok"
The flow runs with no network	one frame with the network removed
Trained trees place edges better than the steepest slope	0.63 against about 1.0 px RMS on held-out painted lines
Whole-line width repeats between shots	0.07-0.14 px SD
Band edges alone cannot be used for defects	they flag 40-87 % of good lines
The two-source deviation rule rarely flags a good line	1 of 257 at threshold 7
Filters before a segmenter do not help	SAM, the local detector and SegFormer, each measured
The trained segmenter reads soft white frames	1 of 17 lines doubtful against 9 of 17
The glue model on `main` is usable	all lines, equal widths by colour, 3 s on the CPU
The width limits behave as designed	9 % and 13 % flagged where 5 % and 15 % were expected
New settings do not change existing behaviour	default detector output byte-identical; 366 tests pass

22.2 Judged by the user

- Version 2: good, black perfect. Version 3: worse than 2. Version 4: best so far.
- Versions 5 and 6, the width report and the main-model comparison have **not** been judged by the user.

22.3 Not verified

Open point	Why it is open
Whether any method catches a real width defect	no defective sample; painted defects are a model
Sudden width changes and steps	no test for them; the mask may round them off
Width in millimetres	camera uncalibrated; the scale is fitted to the nominal 4 mm; no caliper reading
Pink width	version 5 and the model on `main` differ by 2.6 px
White fabric	18 lines in the set, 6 held out
Soft white frames from other sets	only the trained segmenter handled them, on few frames
Line ends	the outline stops short of the bead's tips on some panels
The trained segmenter on another product	not tried; its licence is non-commercial
Local SAM 3	gated weights; not run
The Windows inspection PC	none of the research has run there
Part D of this book	read from source at one commit; only the glue model was run
The application's own cycle time	not re-timed in this work

22.4 Next steps, in order of what they unlock

1. **Defective samples.** A few strips made too wide and too narrow on purpose, sudden and gradual. Nothing about catching defects can be settled without them.
2. **Calibrate the camera**, then caliper widths on a few lines per colour. Settles the scale, the width definition and the pink dispute.
3. **Short windows and a jump test** in `segmentation/width_stats.py`, measured on painted sudden defects; flag a change that the band edges show and the mask does not.
4. **Trees and ribbon after the glue model on `main`.** The best update found for it; it gives the even width the raw reading lacks, on the CPU.
5. **Decide on the trained segmenter**: licence, and retraining per product.
6. **Merge `main` into the working branch** when the two lines are to become one; eleven files need care (chapter 20).
7. For pink: a second light at a low angle.

22.5 Rules for changing this system

- Work on a separate branch, never on `main`. `main` is distributed by the updater.
- Do not commit until the change is validated and asked for.
- New behaviour is opt-in: a settings key whose default reproduces the old behaviour, read with `.get`.
- Do not put `null` in `config.json`.
- `.env` holds the Roboflow key and is git-ignored. A cloud workflow or cloud SAM uploads image tiles.
- Transcripts and session files in the repository folder may contain the key; they are git-ignored and must stay out of commits.

23 Appendices

23.1 File index (branch `tooling/pipeline-lab`)

One line per Python file, taken from the file's own description.

Application, repository root

File	What it is
`app_config.py`	Shared configuration loaded after the startup settings window
`app_storage.py`	Portable storage for user data, separate from application code and config
`camera_handler.py`	Background full-resolution capture, shared by setup and dashboard
`config_validation.py`	Checks every section of `config.json` at start
`crop_processing.py`	Persistent rotated crop geometry shared by setup and inspection
`crop_result_view.py`	Tabbed, zoomable view of the crops an inspection produced
`main_1366.py`	Entry point and main window (`IndustrialDashboard`)
`plane_scale.py`	Millimetre scale for a deskewed crop, from the calibrated measurement plane
`region_calibration.py`	Per-crop-region local homographies, as an alternative to the global plane
`sam_detection.py`	SAM3 object detection through the Roboflow serverless workflow
`startup_settings.py`	Startup countdown and form editor; no camera or app imports before selection
`strip_analysis.py`	Centreline, segmentation and width measurement for a detected strip
`ui_theme.py`	Shared, low-overhead visual system for the desktop application

`auto_trigger/`

File	What it is
`auto_trigger/checkpoint.py`	Hand-cycle checkpoint and debouncer
`auto_trigger/controller.py`	Latest-frame hand monitor with fabric gating and stale-result protection
`auto_trigger/fabric_check.py`	Reference H5 loading and fabric preprocessing, copied from hand_traker
`auto_trigger/models.py`	Synchronous inference adapters derived from hand_traker; called off Tk

`inspection/`

File	What it is
`inspection/dashboard.py`	Tk integration; cameras and calibration remain owned by the existing app
`inspection/measurement.py`	Reuse the established strip algorithm for every automatically found fabric
`inspection/pipeline.py`	One immutable capture -> segmentation -> existing measurements -> saved set
`inspection/result_view.py`	Large, persistent fabric tabs with the established zoom/pan image viewer
`inspection/storage.py`	One capture directory per detection; lossless sources and bounded retention

`segmentation/`

File	What it is
`segmentation/__init__.py`	Replaceable segmentation boundary
`segmentation/bead_edges.py`	The glue bead's own two edges along a roughly known line, from the camera's pixels
`segmentation/bead_ml.py`	Traditional machine learning for the glue band's edges
`segmentation/edge_methods.py`	Ways of finding the glue band's two edges in a straightened strip
`segmentation/glue_line.py`	Local glue-line detector for pale fabric
`segmentation/glue_shape.py`	The known shape of a glue line, used as a condition on what is reported
`segmentation/provider.py`	Reference workflow preprocessing and coordinate mapping, isolated from UI
`segmentation/ribbon.py`	One smooth ribbon of even width from the edge evidence along a glue line
`segmentation/sam_local.py`	SAM on this computer's own GPU: nothing is uploaded
`segmentation/sam_refine.py`	Exact glue-line outline from SAM, prompted with points on the roughly found line
`segmentation/segformer_local.py`	A trained SegFormer that marks the glue band, and the band's width read from its mask
`segmentation/synthetic_bead.py`	Glue lines of known edges, painted onto real fabric: a truth to measure error against
`segmentation/width_stats.py`	Statistics of a glue line's width: millimetres from pixels

`tools/`

File	What it is
`tools/accuracy_bench.py`	How far each way of finding the glue line's edges is from the truth
`tools/benchmark_flow.py`	Where the time goes in a Pipeline Lab run: every stage timed inside `run_job`
`tools/learn_bead_reference.py`	Learn what a glue line looks like from frames where it is seen perfectly
`tools/learn_glue_shape.py`	Learn the wave shape of a product's glue line from sample frames
`tools/pipeline_lab.py`	Pipeline Lab: try pre-processing and segmentation on any image, by hand
`tools/pipeline_steps.py`	Pre-processing steps for the Pipeline Lab, and running them in a chosen order
`tools/pipeline_sweep.py`	Sweep pre-processing recipes against the local glue-line detector, and score them
`tools/replay_capture.py`	Replay saved images through the automatic inspection stages, offline
`tools/roi_sweep.py`	Per-ROI recipe sweep: extract every region the trained model marks, then crop
`tools/train_bead_ml.py`	Train the gradient-boosted edge corrector (`segmentation.bead_ml`) on painted glue lines
`tools/width_report.py`	Width report: where each glue line is wider or narrower than a good line's natural variation allows

`CalibrateAPP/` and `measure/`

File	What it is
`CalibrateAPP/calibration_math.py`	Reusable, testable camera-calibration math helpers
`CalibrateAPP/calibration_store.py`	Replace a calibration's related JSON files, rolling back failed saves
`CalibrateAPP/calibration_ui.py`	Themed two-stage ChArUco calibration for the industrial vision app
`CalibrateAPP/generate_board.py`	Print any saved ChArUco board profile at an exact physical size
`CalibrateAPP/generate_two_boards.py`	Generate the two uniquely numbered ChArUco boards used by two-board mode
`CalibrateAPP/measurement_accuracy.py`	Real-world ChArUco distance verification for a saved measurement plane
`CalibrateAPP/region_calibration_ui.py`	Operator page for the optional per-region homography calibration mode
`CalibrateAPP/verify_calibration_pipeline.py`	Deterministic numerical verification of calibration and plane projection math
`measure/aruco_plane.py`	Per-frame metric measurement plane from one configured ArUco marker
`measure/makeUndistored.py`	-

23.2 Documents

Document	What it covers
`docs/README.md`	index of all documents
`docs/inspection/AUTOMATIC_INSPECTION.md`	operator cycle, providers, the local detector and its settings, Pipeline Lab, saved data
`docs/inspection/CONFIGURATION.md`	cameras, preview, crops, calibration and measurement settings
`docs/inspection/PERFORMANCE.md`	load on a low-power PC
`docs/inspection/UI_DESIGN.md`	the visual system of the windows
`docs/calibration/CALIBRATION_PIPELINE.md`	lens calibration, measurement plane, millimetre scale
`docs/glue-line/GLUE_LINE_FINDINGS.md`	every measurement of Part C, with the methods that failed
`docs/handbook/`	this book: chapters, diagrams, charts, build script
`CLAUDE.md`, `.claude/skills/glue-line-vision/`	layout, rules and glue-line knowledge for coding assistants

23.3 Result pages (outside the repository)

Each is a folder under `~/flod_experiments/out/` with an `index.html`.

Folder	Shows
`lab_validation`, `lab_validation2`	leak repair on 18 validation frames
`v4_m1_all`	version 4 on all 74 frames
`local_gpu_m1_all`	the local GPU flow on all 74 frames
`v5_m1_all`, `v5b_m1_all`	version 5 on all 74 frames
`compare_stages`	versions 2 to 5 on the same lines
`segformer_vs_sam`	the trained segmenter against version 5
`main_model_check`	the glue model on `main`, with and without trees and ribbon
`width_report`	width in mm along every line, changes and glitches

23.4 Environments

Environment	Python	Used for
Inspection PC, project `.venv`	3.12	the application: NumPy, OpenCV, MediaPipe, TensorFlow / tf-keras, inference-sdk
`~/flod_experiments/gpu`	3.12	research: PyTorch 2.11 with CUDA, transformers, onnxruntime, ultralytics, scikit-learn, lightgbm; also `markdown` and `matplotlib` for this book
System Python on the development PC	3.14	lacks several dependencies; not used

23.5 Rebuilding this book

```
python docs/handbook/build.py
```

- Text: `docs/handbook/chapters/*.md`. Diagrams: `docs/handbook/diagrams/*.dot` (Graphviz). Charts: `docs/handbook/charts.py`. Layout: `docs/handbook/style.css`.
- Needs the `markdown` package, `matplotlib`, Graphviz and Google Chrome or Chromium.
- The example pictures in `figures/*.jpg` are committed. `pictures.py` remakes them and needs the sample frames and result pages, which are not in the repository.
- Chapter, section and figure numbers and the contents list are made by the build.